
# AI AGENTS FOR HACKING

**OMAR SANTOS**

@santosomar

santosomar

hackerrepo.org

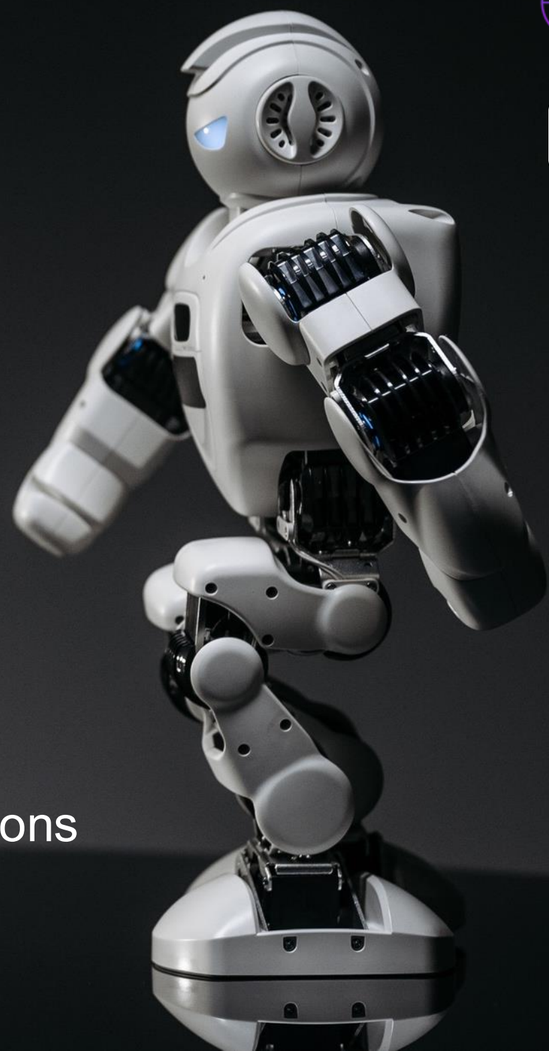
h4cker.org

# AGENDA

## SEGMENT 1

Introduction to RAG in Cybersecurity

- Course overview and objectives
- Understanding the LLM stack and relevancy for cybersecurity operations
- Embeddings and Embedding Models
- Indexing Techniques
- Vector Databases
- Chunking strategies
- RAG vs. Fine-tuning
- Good old, RAG, RAG Fusion, and RAPTOR
- Agentic RAG
- Model Context Protocol (MCP)
- Hands-on labs: Embeddings and basic inference implementations with OpenAI API and Claude API using cybersecurity data.




hackerrepo.org

santosomar

h4cker.org

@santosomar

# AGENDA
## SEGMENT 2

hackerrepo.org
santosomar
h4cker.org
@santosomar

Introducing LangChain, LangGraph, and LLamaIndex

- Introducing LangChain
- LangChain vs. LlamaIndex
- Prompt templates and system prompts
- LangServe and LangSmith
- AI agent frameworks and LangGraph
- Hands-on Lab: Using LangChain and Prompt Templates for cybersecurity and network operations

# AGENDA
## SEGMENT 3

hackerrepo.org

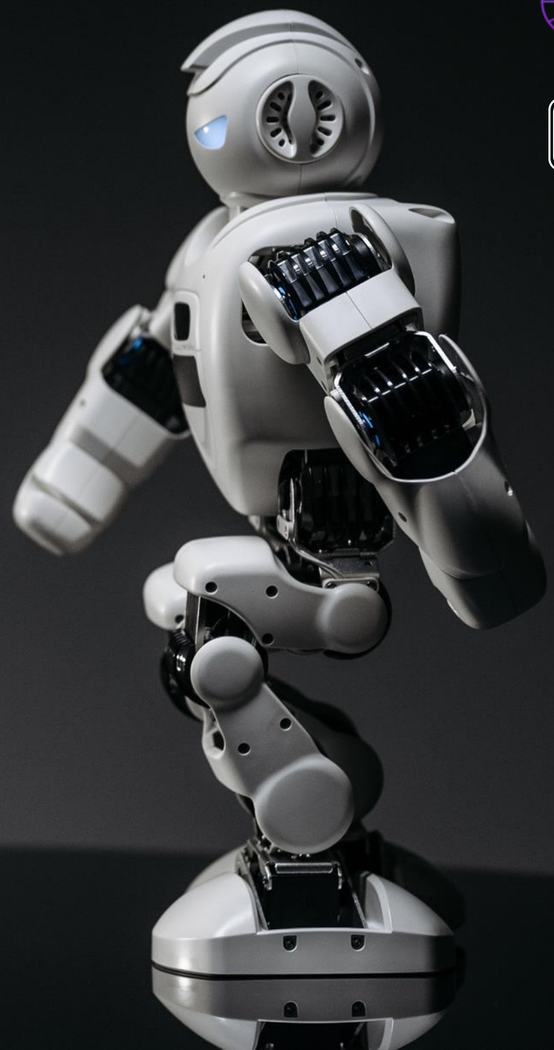
santosomar

h4cker.org

@santosomar

Prompt Chaining and Retrieval Augmented Generation Labs for Cybersecurity and Networking Professionals

- Basic Chain Examples
- Branching Chains
- Parallel Chains
- Hands-on labs: Basic, Branching, and Parallel Chains
- Hands-on labs: Basic RAG, one-off-questions, metadata, text splitting, and web scraping for passive reconnaissance and open-source intelligence (OSINT)



# AGENDA
## SEGMENT 4

hackerrepo.org

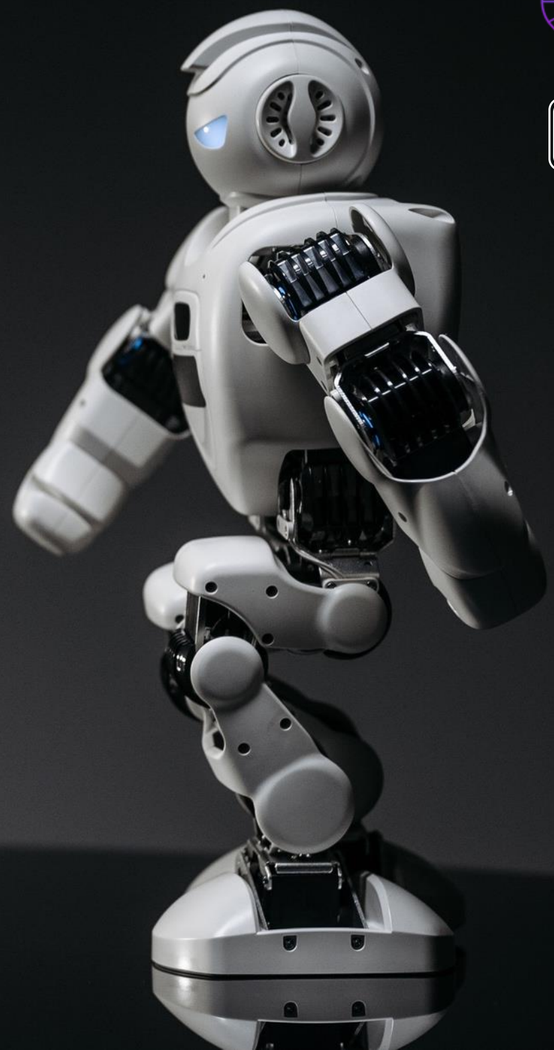
santosomar

h4cker.org

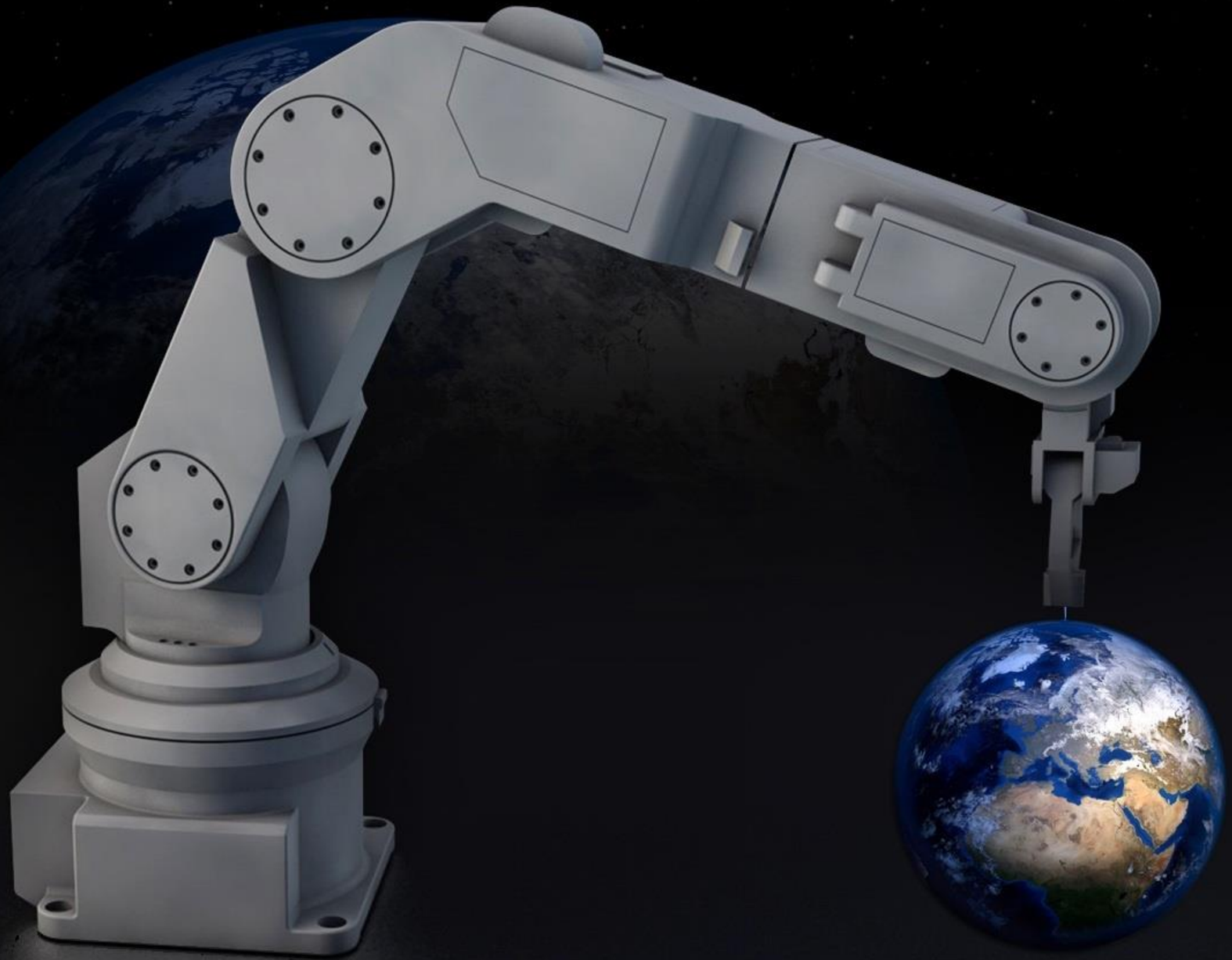
@santosomar

AI Agents and Agentic Frameworks

- Introduction to AI agents and related frameworks
- Introducing LangGraph
- Surveying LangGraph Cloud
- Function calling and tool calling
- Agents and the Model Context Protocol (MCP) Servers/Clients
- Hands-on lab: A basic agent using function and tool calling for ethical hacking



# COURSE OVERVIEW AND OBJECTIVES

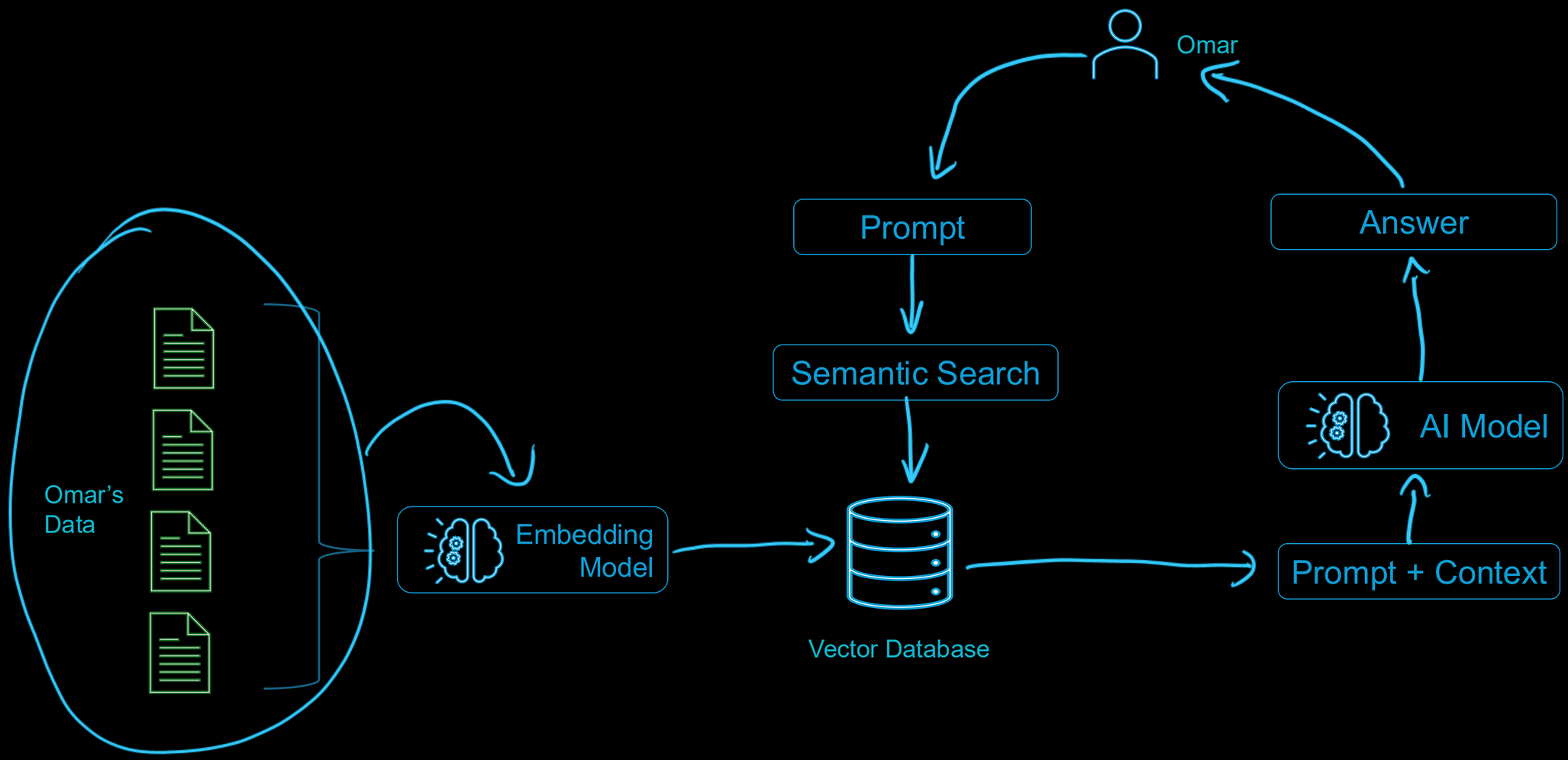


https://hackerrepo.org

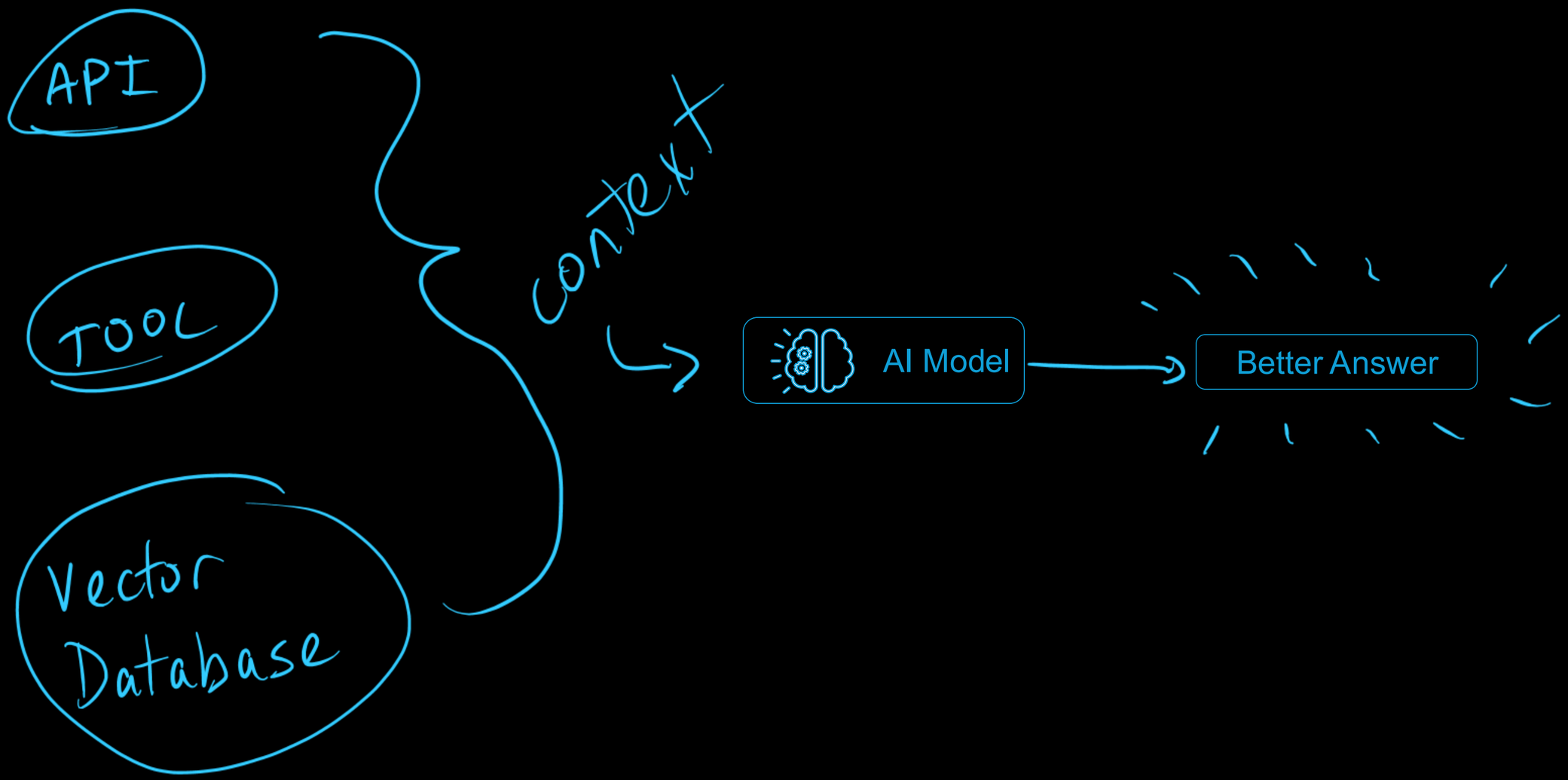
https://hackingwithai.org

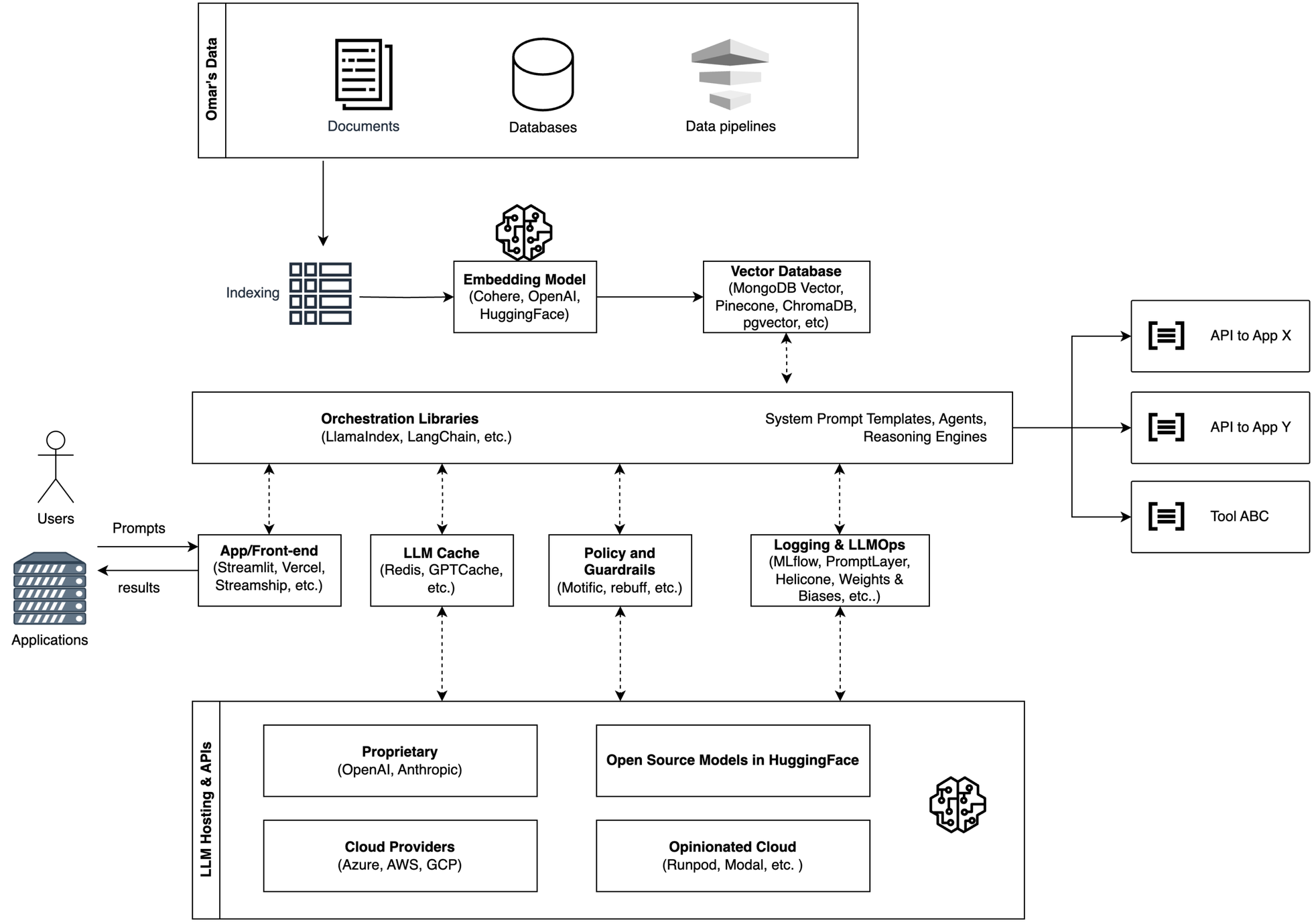
TWO GITHUB REPOS FOR THIS COURSE

# What is Retrieval Augmented Generation (RAG)?



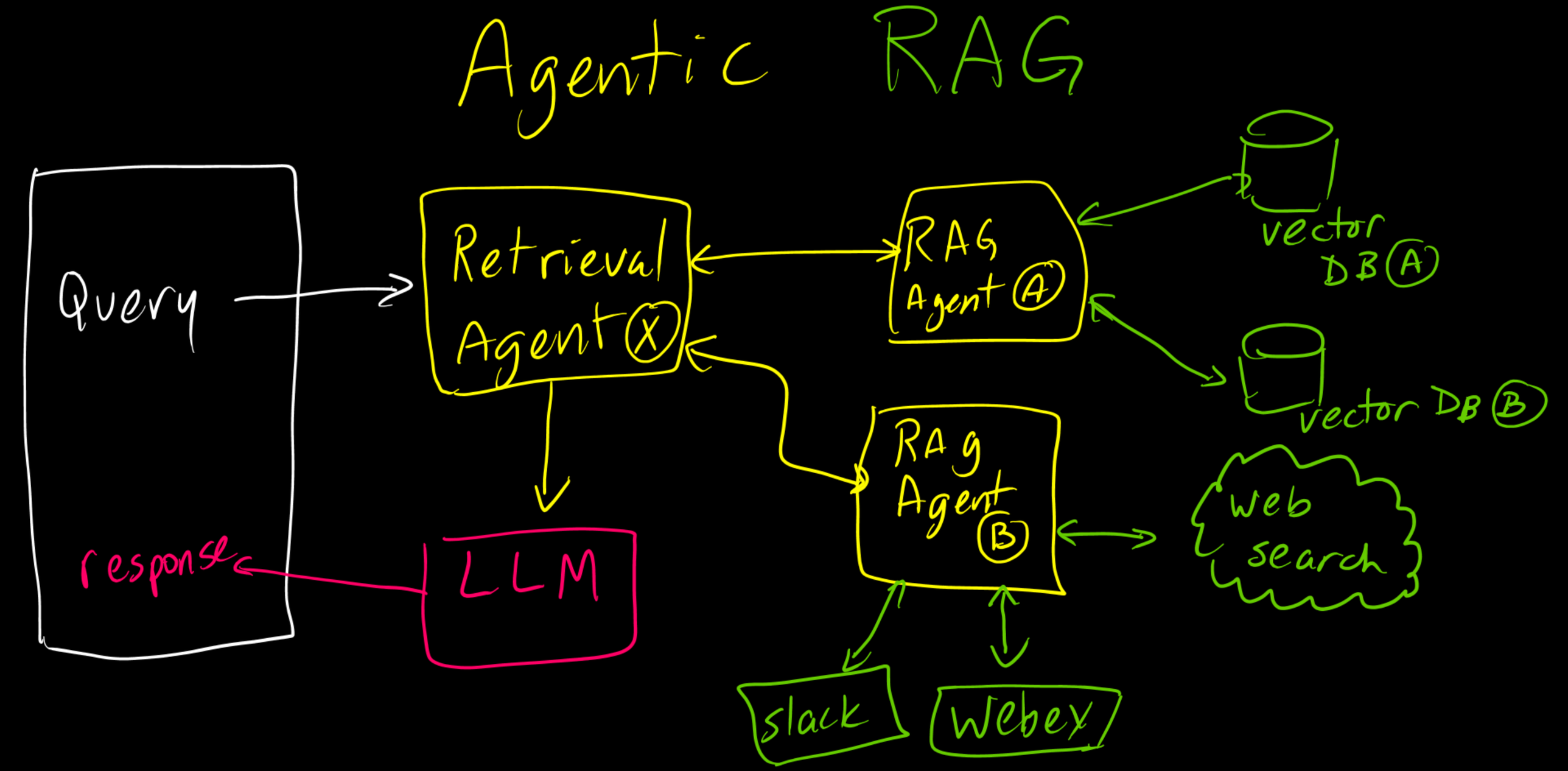

https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_05


API
TOOL
Vector Database
context
AI Model
Better Answer

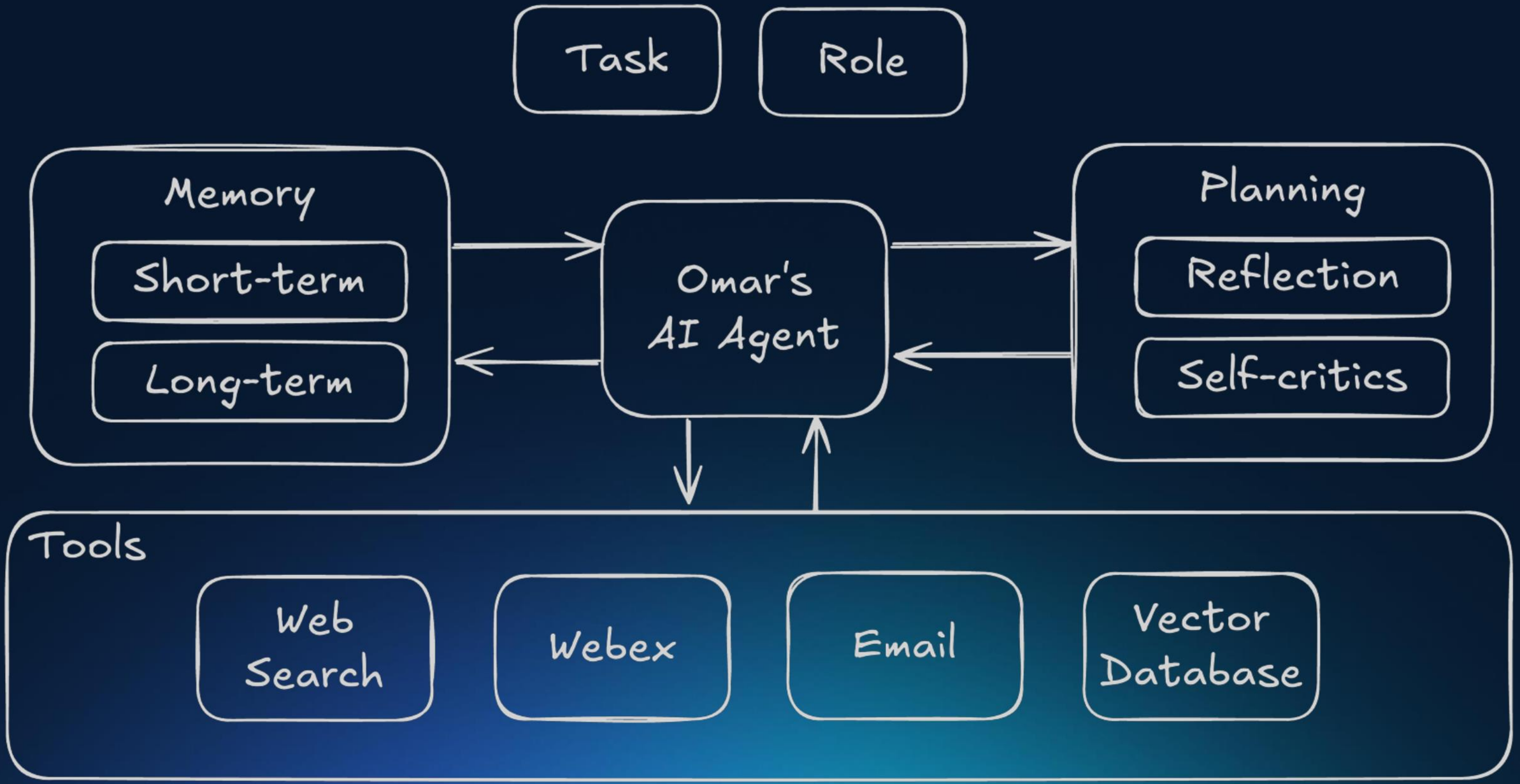

Omar's Data
Documents
Databases
Data pipelines
Indexing
Embedding Model
(Cohere, OpenAI, HuggingFace)
Vector Database
(MongoDB Vector, Pinecone, ChromaDB, pgvector, etc)
API to App X
API to App Y
Tool ABC
Orchestration Libraries
(LlamaIndex, LangChain, etc.)
System Prompt Templates, Agents, Reasoning Engines
Users
Prompts
results
Applications
App/Front-end
(Streamlit, Vercel, Streamship, etc.)
LLM Cache
(Redis, GPTCache, etc.)
Policy and Guardrails
(Motific, rebuff, etc.)
Logging & LLMOps
(MLflow, PromptLayer, Helicone, Weights & Biases, etc..)
LLM Hosting & APIs
Proprietary
(OpenAI, Anthropic)
Open Source Models in HuggingFace
Cloud Providers
(Azure, AWS, GCP)
Opinionated Cloud
(Runpod, Modal, etc. )

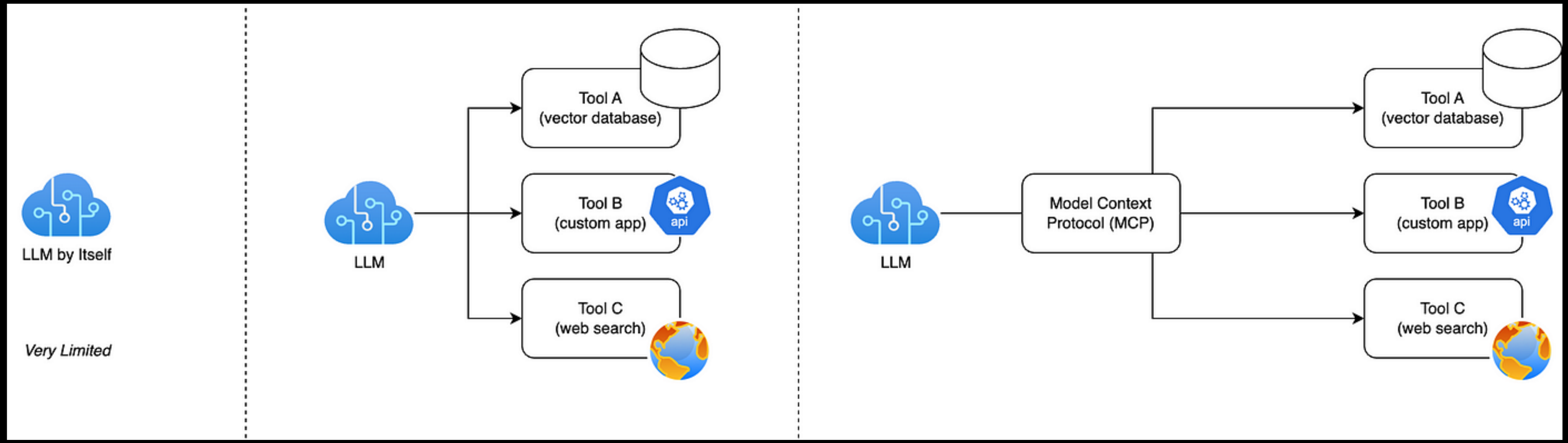
# RAG vs. Fine-Tuning

| | RAG | Fine-Tuning |
|---|---|---|
| Data Source | External/internal databases (real-time) | Specialized, static datasets |
| Cost | Lower (no retraining required) | Higher (requires retraining) |
| Response Accuracy | High for factual, real-time queries | High for domain-specific tasks |
| Use Case | Dynamic environments (e.g., customer support, dynamic data, etc.) | Static environments (e.g., legal or medical). Domain specific tasks. |
| Computational Requirements | Lower | Higher during fine-tuning. |
| Prone to Hallucinations | Less likely due to real-time retrieval, but can still hallucinate depending on the implementation | Can still hallucinate if faced with unfamiliar data |



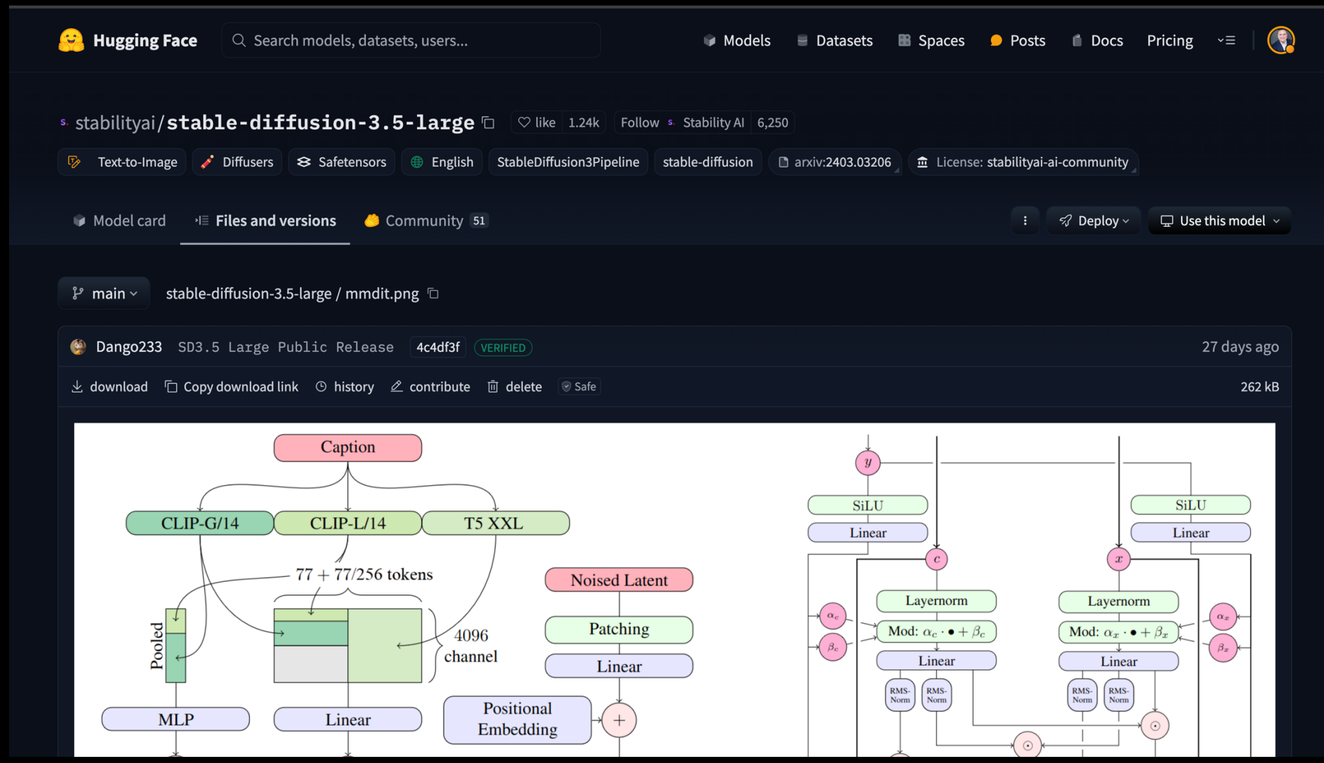

https://becomingahacker.org/integrating-agentic-rag-with-mcp-servers-technical-implementation-guide-1aba8fd4e442


Task
Role
Memory
Short-term
Long-term
Omar's
AI Agent
Planning
Reflection
Self-critics
Tools
Web
Search
Webex
Email
Vector
Database




https://becomingahacker.org/integrating-agentic-rag-with-mcp-servers-technical-implementation-guide-1aba8fd4e442

# An Overview of Open-source / Open-Weight AI Models and Hugging Face

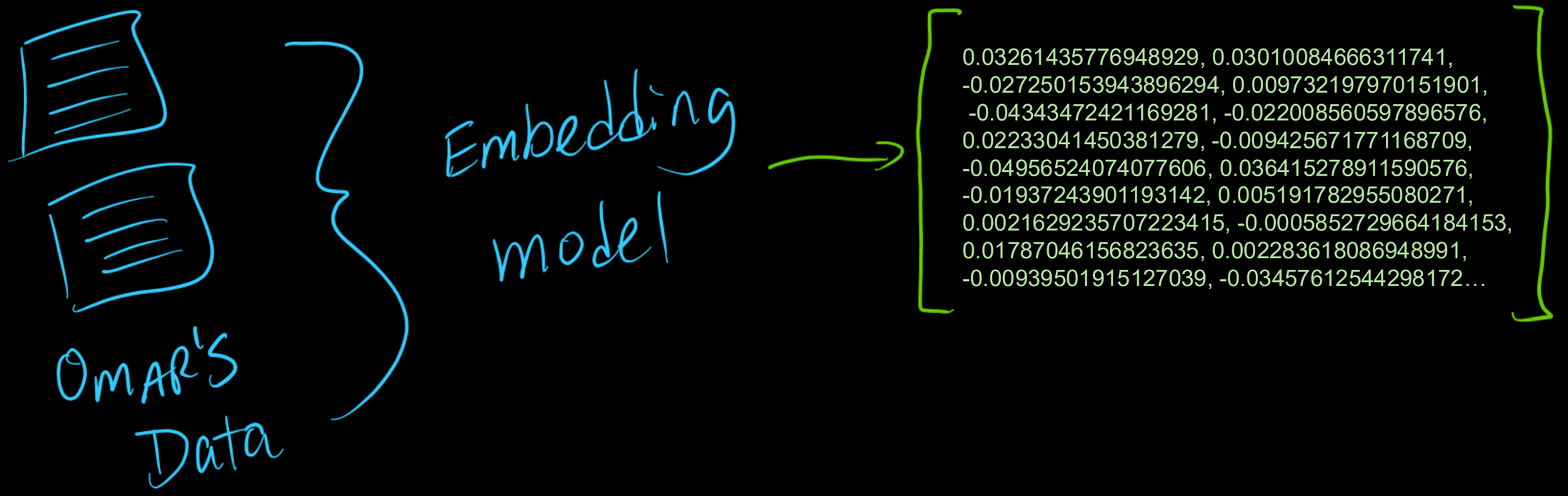



https://becomingahacker.org/a94502e3d5f12

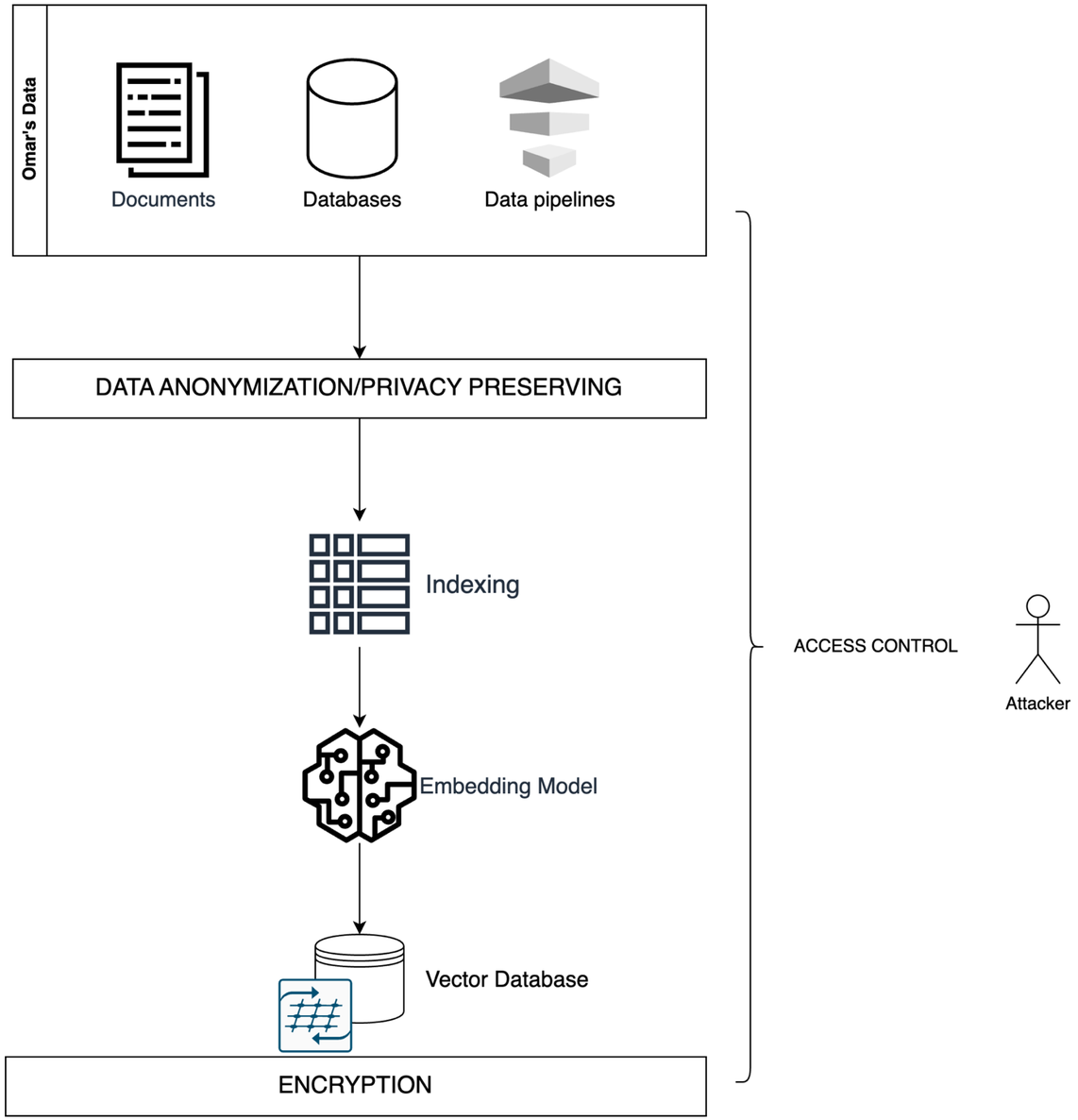
# Embeddings and Embedding Models

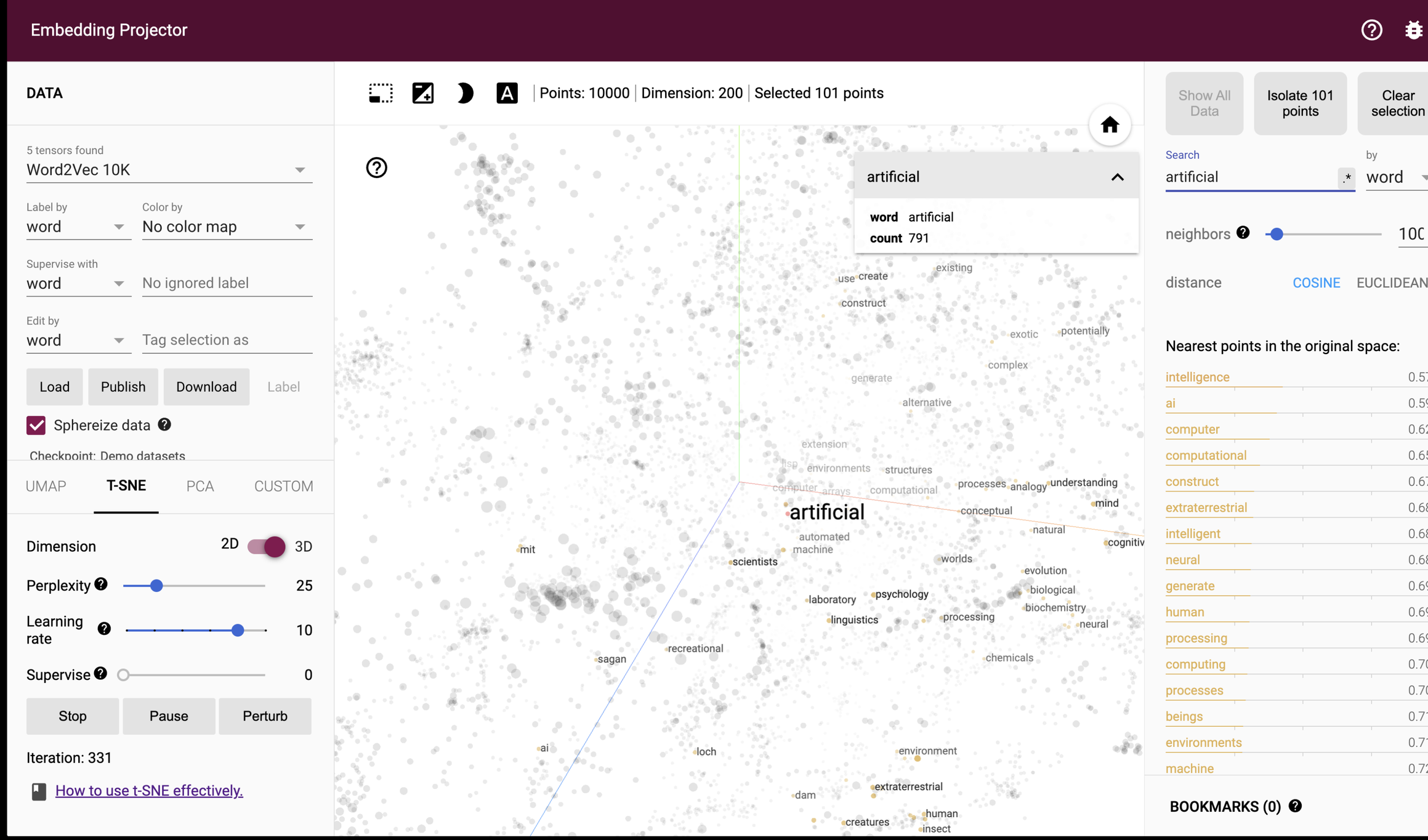
Embeddings are dense numerical representations of real-world objects, such as words, images, or audio, that allow machine learning models to process and understand complex data.
These representations are typically expressed as vectors in a multi-dimensional space, with each dimension capturing a specific feature or characteristic of the object.

The primary purpose of embeddings is to transform high-dimensional data into a lower-dimensional space while preserving the most relevant information, making it easier for machine learning algorithms to work with and identify patterns or similarities between objects.




https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_06


Omar's Data
Documents
Databases
Data pipelines
DATA ANONYMIZATION/PRIVACY PRESERVING
Indexing
Embedding Model
Vector Database
ENCRYPTION
ACCESS CONTROL
Attacker

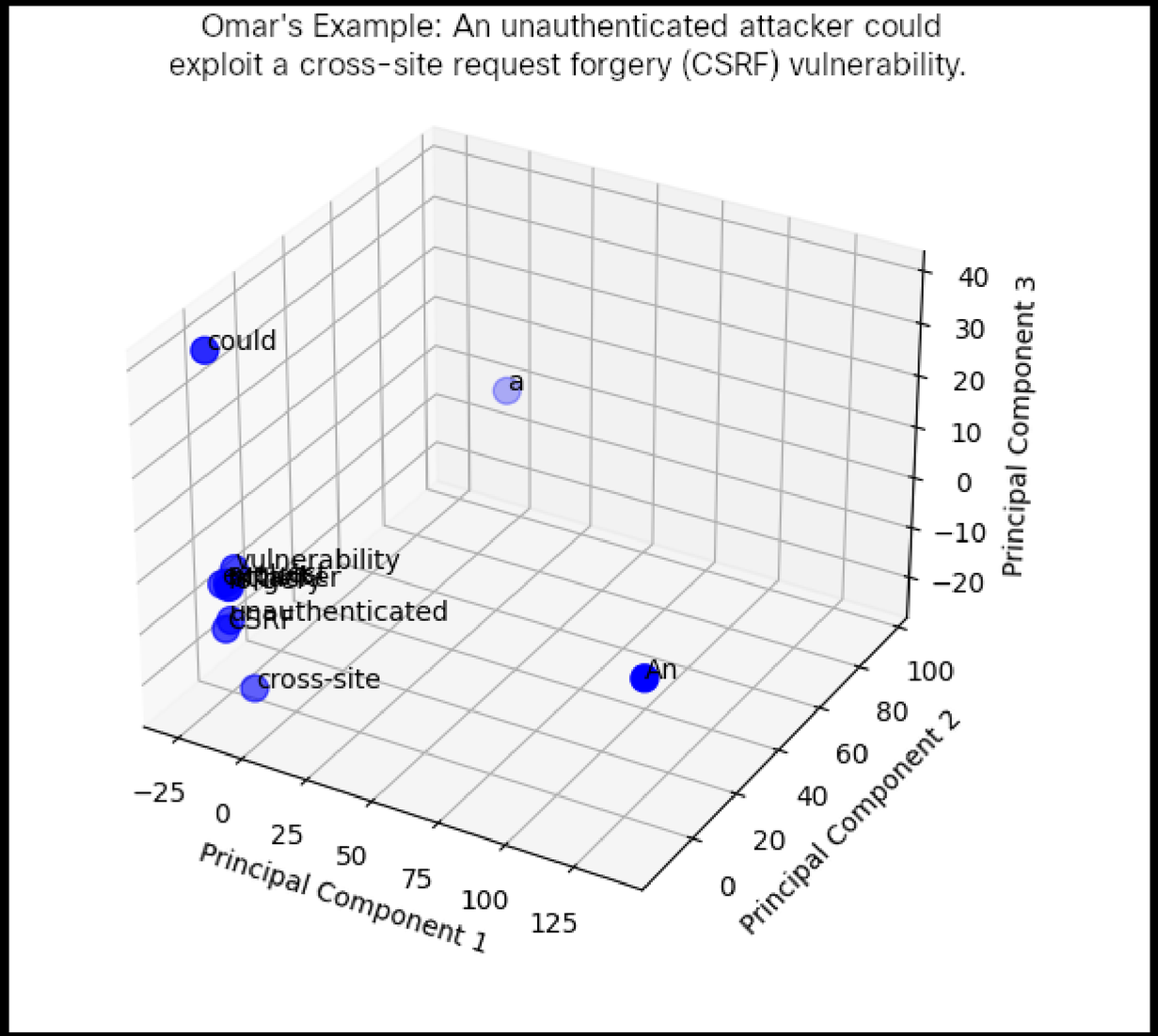

Embedding Projector
DATA
5 tensors found
Word2Vec 10K
Label by
word
Color by
No color map
Supervise with
word
No ignored label
Edit by
word
Tag selection as
Load
Publish
Download
Label
Sphereize data
Checkpoint: Demo datasets
UMAP
T-SNE
PCA
CUSTOM
Dimension
2D
3D
Perplexity
25
Learning rate
10
Supervise
0
Stop
Pause
Perturb
Iteration: 331
How to use t-SNE effectively.
Points: 10000 | Dimension: 200 | Selected 101 points
artificial
word artificial
count 791
use
create
existing
construct
exotic
potentially
complex
generate
alternative
extension
lisp
environments
structures
computer
arrays
computational
processes
analogy
understanding
mind
conceptual
natural
cognitiv
automated
machine
mit
scientists
worlds
evolution
biological
laboratory
psychology
biochemistry
linguistics
processing
neural
recreational
sagan
chemicals
ai
loch
environment
extraterrestrial
dam
creatures
human
insect
Show All Data
Isolate 101 points
Clear selection
Search
artificial
.*
by
word
neighbors
100
distance
COSINE
EUCLIDEAN
Nearest points in the original space:
intelligence 0.572
ai 0.594
computer 0.620
computational 0.656
construct 0.676
extraterrestrial 0.680
intelligent 0.686
neural 0.689
generate 0.691
human 0.694
processing 0.697
computing 0.702
processes 0.708
beings 0.717
environments 0.719
machine 0.722
BOOKMARKS (0)
https://projector.tensorflow.org

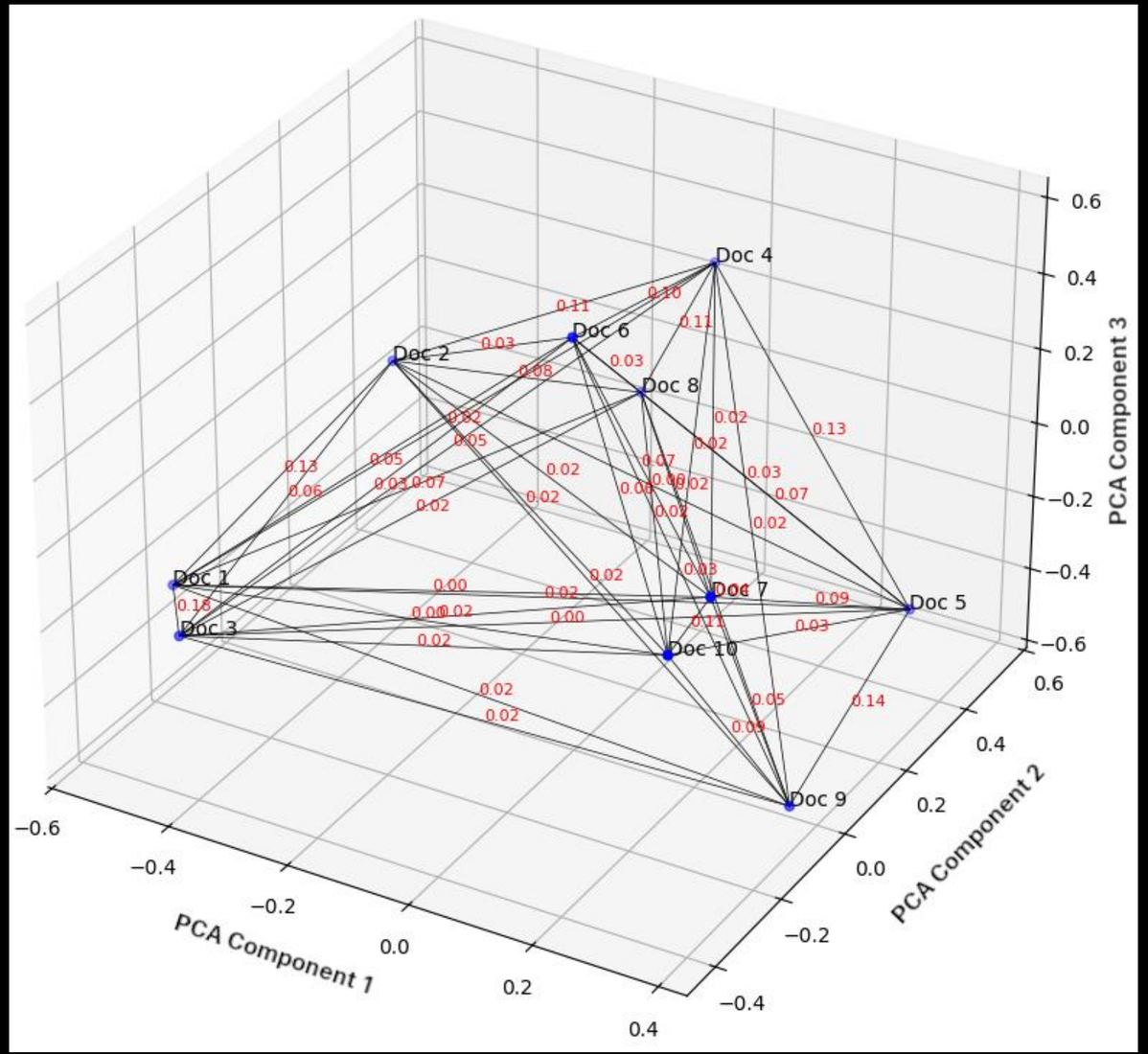
# OpenAI Embeddings Example

```python
# Import the required libraries
from openai import OpenAI
client = OpenAI()

# Create embeddings for the input text using the "text-embedding-3-small" model
response = client.embeddings.create(
    input="This is an example from Omar.",
    model="text-embedding-3-small"
)

# Print the embeddings generated for the input text
print(response.data[0].embedding)
```

https://github.com/santosomar/RAG-for-cybersecurity/blob/main/part1_basic_langchain_openai_no_rag/embeddings.py

https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_06


Omar's Example: An unauthenticated attacker could exploit a cross-site request forgery (CSRF) vulnerability.
could
a
vulnerability
unauthenticated
CSRF
cross-site
An
Principal Component 1
Principal Component 2
Principal Component 3


Doc 1
Doc 2
Doc 3
Doc 4
Doc 5
Doc 6
Doc 7
Doc 8
Doc 9
Doc 10
PCA Component 1
PCA Component 2
PCA Component 3

cohere playground

CHAT DASHBOARD **PLAYGROUND** DOCS COMMUNITY

Try Command R+ in any of its 10 supported languages in the Chat playground!

API request builder | Chat | Classify | Embed | Generate LEGACY

View code | Save | Share

Examples NEURIPS22 ACCEPTED PAPERS | YCOMBINATOR THREAD TITLES | SUBREDDIT TITLES | See more

INPUT

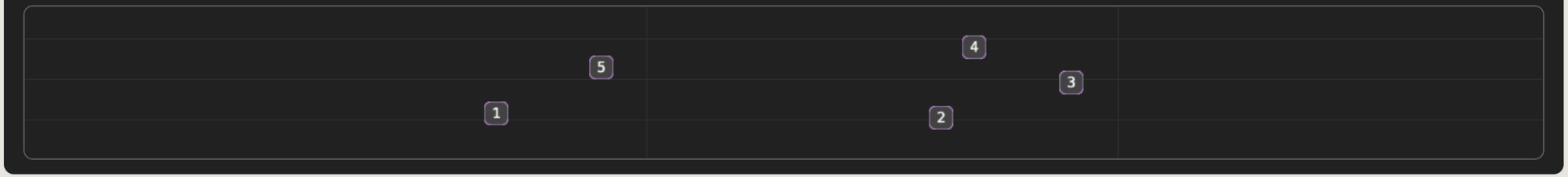
or Upload a file

Embed takes a list of sentences and outputs a list of vectors, allowing models to compare their meanings.

1. This is an example from Omar about a Cybersecurity-related sentence.
2. This is another example about a sentence about a vulnerability like SQL injection.
3. SQL Injection (SQLi) is a type of security vulnerability that allows an attacker to interfere with the queries an application makes to its database.
4. SQL injection occurs when user input is included in an SQL query without proper validation or escaping.
5. Attackers can input malicious SQL code, which gets executed on the database.

OUTPUT

Every point is an input sentence from above. Can you tell that sentences with similar meaning are closer together?




PARAMETERS CODE

Python

```
import cohere
co = cohere.Client('4xssnh6XGEoocUImJ
response = co.embed(
  model='embed-english-light-v3.0',
  texts=["This is an example from Omar abou
  input_type='classification',
  truncate='NONE'
)
print('Embeddings: {}'.format(response.embe
```

CONTROL PANEL

RUN

CLEAR

https://dashboard.cohere.com/playground/embed

# Massive Text Embedding Benchmark (MTEB)

Overall | Bitext Mining | Classification | Clustering | Pair Classification | Reranking | Retrieval | STS | Summarization | MultilabelClassification | Retrieval w/Instructions

English | Chinese | French | Polish | Russian

**Overall MTEB English leaderboard** 🔮

- **Metric:** Various, refer to task tabs
- **Languages:** English

| Rank | Model | Model Size (Million Parameters) | Memory Usage (GB, fp32) | Embedding Dimensions | Max Tokens | Average (56 datasets) | Classification Average (12 datasets) | Clustering Average (11 datasets) |
|---|---|---|---|---|---|---|---|---|
| 1 | NV-Embed-v2 | 7851 | 29.25 | 4096 | 32768 | 72.31 | 90.37 | 58.46 |
| 2 | bge-en-icl | 7111 | 26.49 | 4096 | 32768 | 71.67 | 88.95 | 57.89 |
| 3 | stella_en_1.5B_v5 | 1543 | 5.75 | 8192 | 131072 | 71.19 | 87.63 | 57.69 |
| 4 | SFR-Embedding-2_R | 7111 | 26.49 | 4096 | 32768 | 70.31 | 89.05 | 56.17 |
| 5 | gte-Qwen2-7B-instruct | 7613 | 28.36 | 3584 | 131072 | 70.24 | 86.58 | 56.92 |
| 6 | gte-Qwen2-7B-instruct-GGUF | | | | | 70.24 | 86.58 | 56.92 |
| 7 | stella_en_400M_v5 | 435 | 1.62 | 8192 | 8192 | 70.11 | 86.67 | 56.7 |
| 8 | bge-multilingual-gemma2 | 9242 | 34.43 | 3584 | 8192 | 69.88 | 88.08 | 54.65 |

https://huggingface.co/spaces/mteb/leaderboard

# MTEB

**Massive Text Embedding Benchmark**

**8 Tasks**

**58 Datasets**

## Clustering

- ArxivP2P
- ArxivS2S
- BiorxivP2P
- BiorxivS2S
- MedrxivP2P
- MedrxivS2S
- Reddit
- RedditP2P
- StackExchange
- StackExchangeP2P
- TwentyNewsgroup

## Bitext Mining

- BUCC
- Tatoeba

## Retrieval


- ArguAna
- ClimateFEVER
- DBPedia
- CQADupstackRetrieval
- FEVER
- FiQA2018
- HotpotQA
- MSMARCO
- NFCorpus
- NQ
- Quora
- SCIDOCS
- SciFact
- Touche2020
- TRECCOVID

## STS

- BIOSESS
- SICK-R
- STS11
- STS12
- STS13
- STS14
- STS15
- STS16
- STS17
- STS22
- STSB

## Summarization

- SummEval

## Classification

- AmazonCounterfactual
- AmazonPolarity
- AmazonReviews
- Banking77
- Emotion
- Imdb
- MassiveIntent
- MassiveScenario
- MTOPDomain
- MTOPIntent
- ToxicConversations
- TweetSentimentExtraction

## Pair Classification

- SprintDuplicateQuestions
- TwitterSemEval2015
- TwitterURLCorpus

## Reranking

- AskUbuntuDupQuestions
- MindSmallReranking
- SciDocsRR
- StackOverFlowDupQuestions

https://arxiv.org/pdf/2210.07316

# Fundamentals of AI Security

### What are AI Security Vulnerabilities?

Cisco defines AI and machine learning security vulnerabilities as an exploitable weakness in an AI model, related software, or hardware code that negatively affects confidentiality, integrity or availability.

Read more...

### Securing Vector Databases

Vector databases are commonly used in Artificial Intelligence and machine learning, and as AI popularity continues to grow, securing such databases is becoming more critical.

Read more...

### AI Training Environment Security

Implementing AI training environments requires many considerations, and security should be one of the foremost. Familiarity of a specific set of security best practices is a must when setting up these environments.

Read more...

### Lifecycle Security and AI Systems

Securing AI systems is an ongoing process, one that stretches the entire lifecycle of the system's AI model and applications. It is important to approach each phase of the lifecycle with its unique needs in mind.

Read more...

### Reference Architectures

Large language models (LLMs) are susceptible to their own set of security risks. It is vital for developers of applications using LLMs to adhere to strict design practices to reduce exposure to these risks.

Read more...

### Selecting Embedding Models

As AI continues to influence content creation, the embedding models necessary for this content and the security of those models and guidelines for embedding model selection become increasingly important.

Read more...

https://aisecurity.cisco.com

# BEST PRACTICES FOR HANDLING CONFIDENTIAL DATA

## CHOOSE YOUR DEPLOYMENT

Consider using embedding models that can be deployed on your own infrastructure or private clouds so that you can maintain full control over data processing.

## USE APPROVED SOLUTIONS

Only use embedding model solutions that are explicitly approved by your company for confidential data processing.

## ANONYMIZE DATA

When possible, anonymize or pseudonymize data before sending it for embedding.

## CONTROL ACCESS

Implement strong access control mechanisms to make sure that only authorized personnel can access and generate embeddings.


## AUDIT LOGS REGULARLY

Conduct regular audits of your data processing pipeline to ensure compliance with corporate policies and regulations.

## REVIEW CONTRACTUAL SAFEGUARDS

If using external services, ensure strong contractual agreements are in place to protect your data and prevent unauthorized use.

# Trade-offs During Embedding Development

| | Pros | Cons |
|---|---|---|
| Embedding Entire Documents | Minimal storage costs | Poor search performance |
| | Simple implementation | Inadequate for large corpora |
| Chunking | Improved search accuracy | Increased storage costs |
| | Better user experience | Higher computational demands |
| Embedding Questions | Exceptional search accuracy | Significant storage costs |
| | Rich semantic representation | Complex implementation |

# Finding the Balance

Determine the required level of search accuracy based on user needs

Evaluate the budget for storage and compute resources

Consider hybrid approaches:

- **Selective Chunking:** Only chunk and generate questions for critical documents
- **Dimensionality Reduction:** Use lower-dimensional embeddings where possible
- **Compression Techniques:** Apply data compression to reduct storage footprint

Vector Databases

- Chroma DB
- pg vector
- Pinecone
- MongoDB Atlas Vector Search
- FIASS
- Milvus
- Weaviate

https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_07

# Vector Indexing Techniques

# Flat Indexing (Brute Force)

Storing vectors without any modifications and performs an exhaustive search by comparing the query vector with every vector in the dataset.

Use Cases: Small-scale databases, benchmarking, or low-dimensional data

Good Accuracy

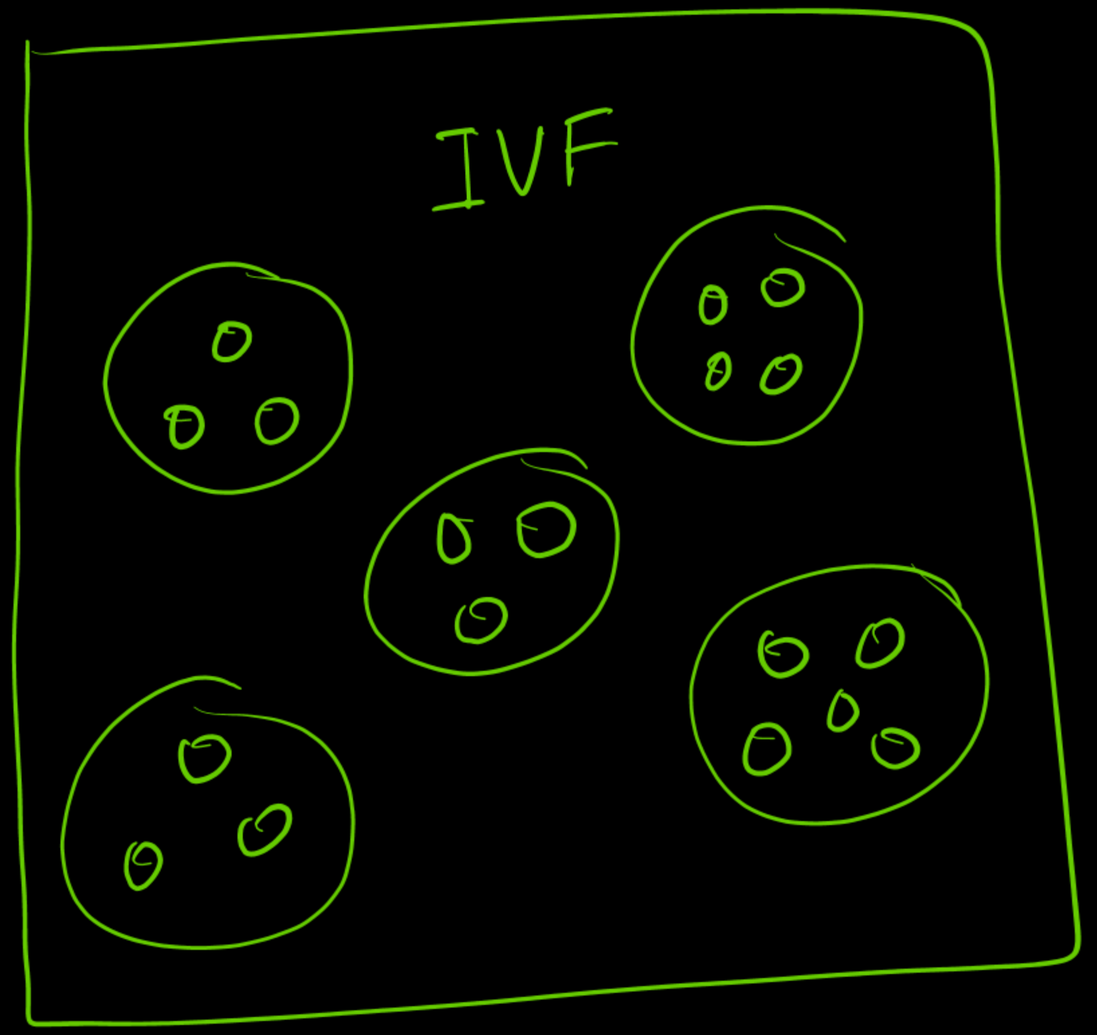
Every vector is evaluated

Slow

Only good for small datasets

# Inverted File Index (IVF)

Vectors are grouped into clusters using techniques like K-means clustering. When a query arrives, only the relevant cluster(s) are searched, reducing the number of comparisons.




$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 = \arg\min_{\mathbf{S}} \sum_{i=1}^{k} |S_i| \operatorname{Var} S_i$$

where $\boldsymbol{\mu}_i$ is the mean (also called centroid) of points in $S_i$, i.e.

$$\boldsymbol{\mu}_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x},$$

$|S_i|$ is the size of $S_i$, and $\|\cdot\|$ is the usual $L^2$ norm . This is equivalent to minimizing the pairwise squared deviations of points in the same cluster:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \frac{1}{|S_i|} \sum_{\mathbf{x}, \mathbf{y} \in S_i} \|\mathbf{x} - \mathbf{y}\|^2$$

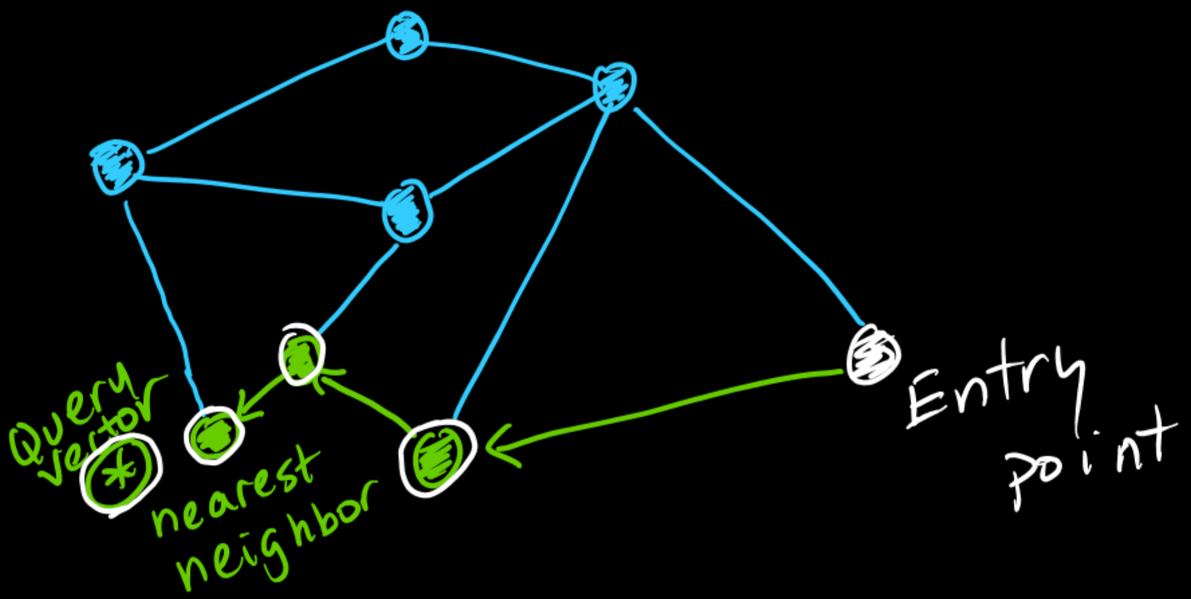
# IVF Advantages and Drawbacks

(+) Faster than Brute Force because it narrows down the search space to specific clusters.

(−) The clustering process can introduce some loss in accuracy.

# Hierarchical Navigable Small World (HNSW) Graph

This graph-based method organizes vectors into multiple layers where each layer contains increasingly refined subsets of vectors. It uses shortcuts to navigate through layers efficiently.





Significantly reduces the number of distance calculations, offering fast retrieval times.

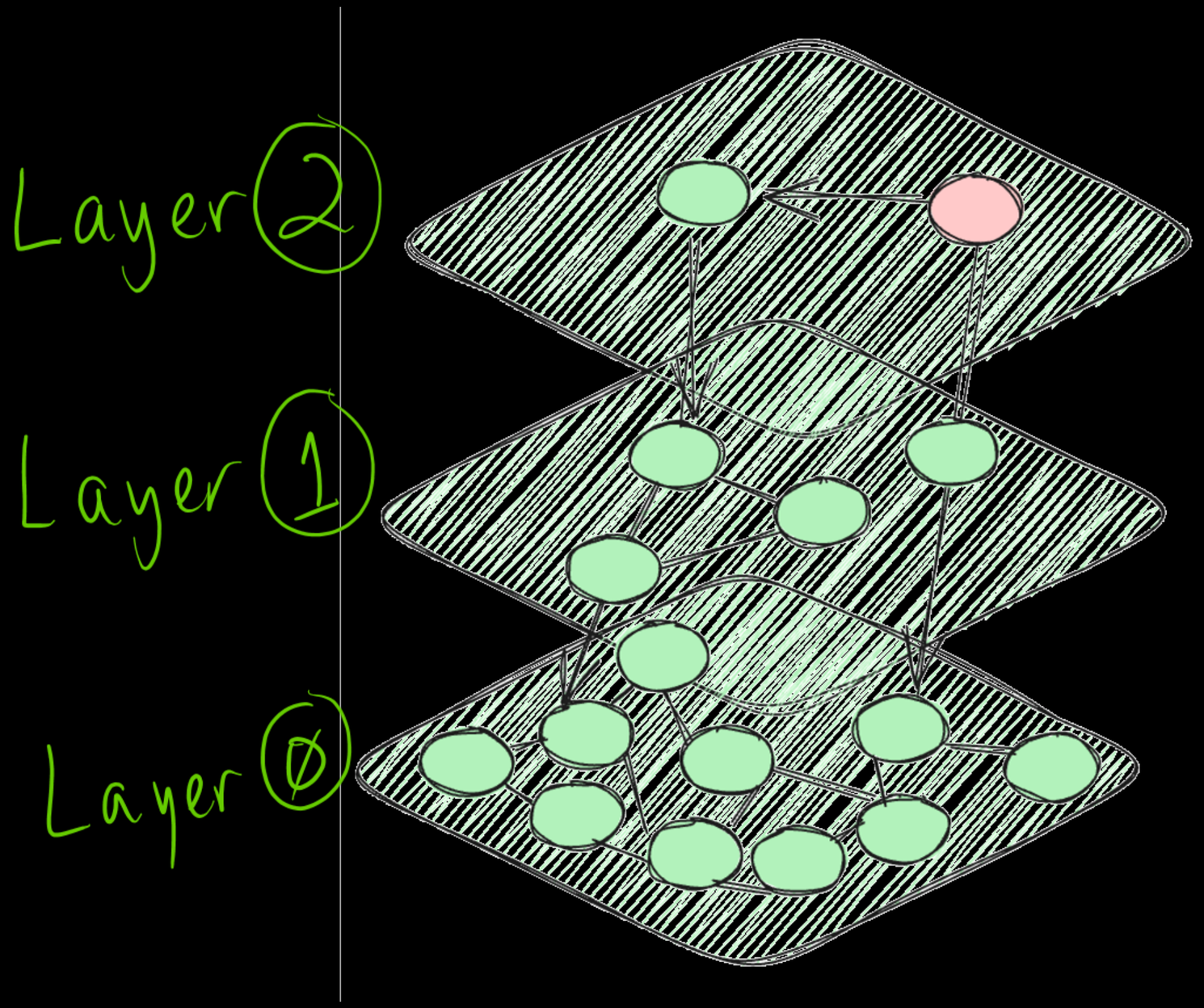

Building the graph can be memory-intensive, especially for large datasets.

# HNSW Layers

The nodes in each layer are linked not only to the nodes within the same layer but also to those in the layers below.

There are fewer nodes at the top, and their density increases as we move down through the layers.

The final layer holds all the data points from the database.



# Other Methods

## Locality Sensitive Hashing (LSH)

- LSH uses hashing to map similar vectors into the same hash bucket with high probability.
- Only vectors in the same bucket are compared during a search.
- Can lead to lower accuracy compared to other methods like IVF or HNSW.

## qFlat Indexing

- A variant of flat indexing that stores vectors on disk rather than in memory, allowing for larger datasets without memory constraints.
- Search speed is dependent on disk access speeds, which can be slower than in-memory searches.

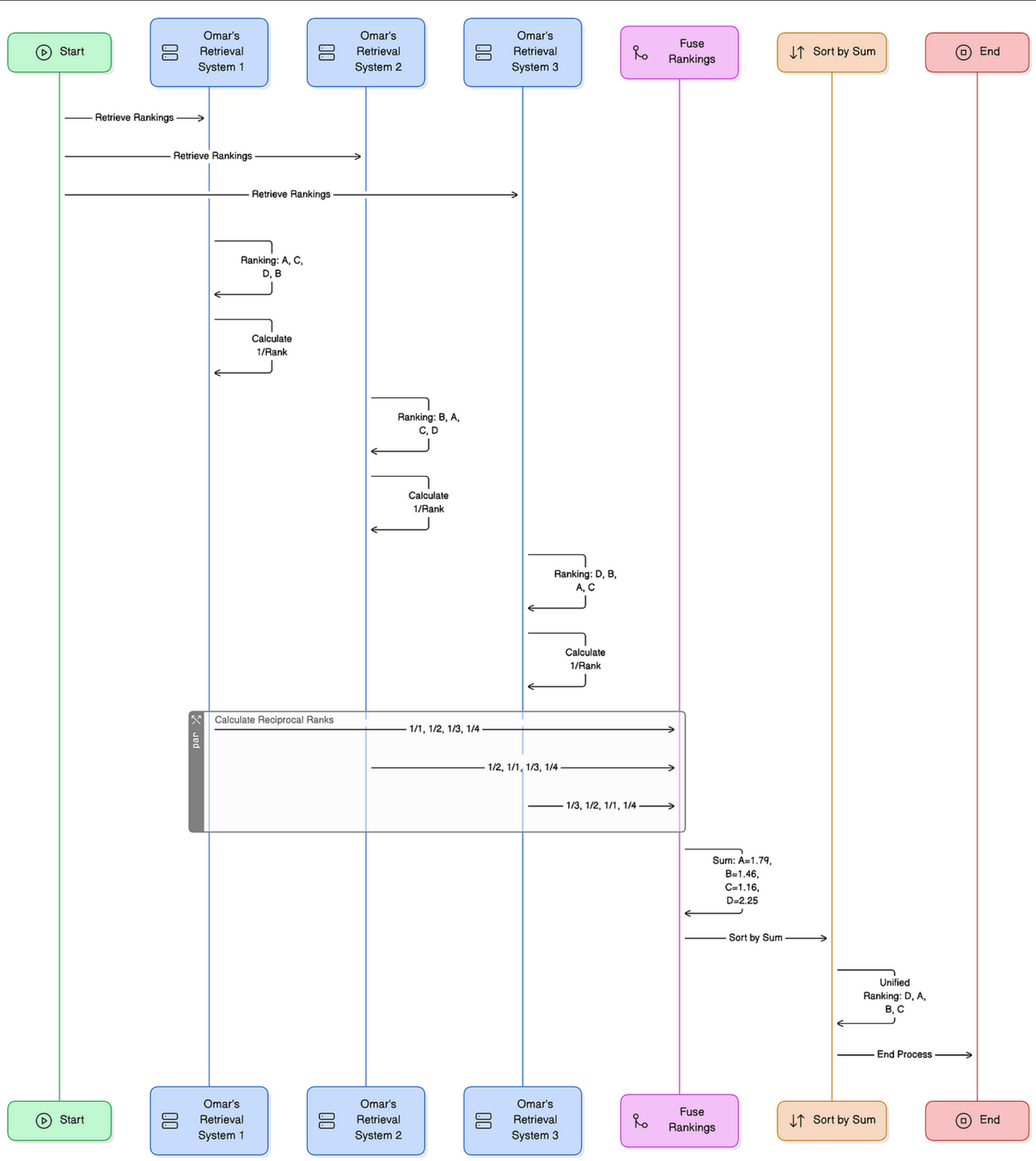
## Multi-Scale Tree Graph (MSTG) Algorithm

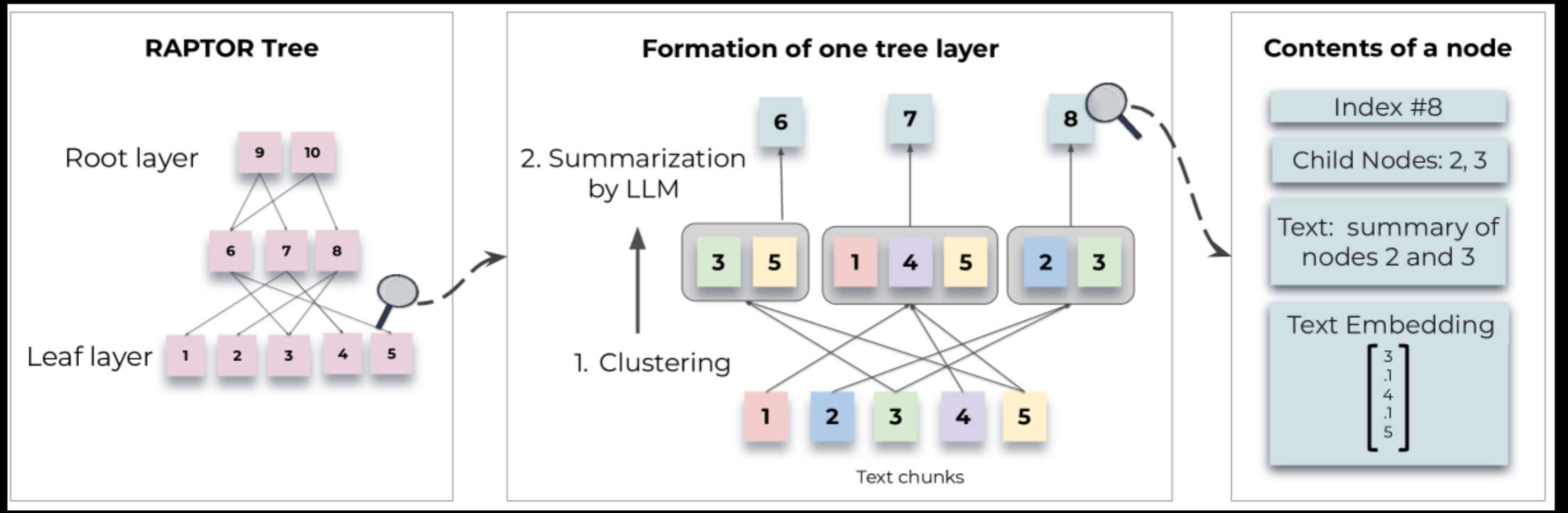
- Combines aspects of tree and graph structures to handle large datasets efficiently by partitioning data into clusters and using hierarchical graphs for navigation.
- Efficiently manages large datasets while balancing memory usage and search performance.
- Can be complex to implement and manage

# Comparing RAG, RAG Fusion, with RAPTOR: Different AI Retrieval-Augmented Implementations

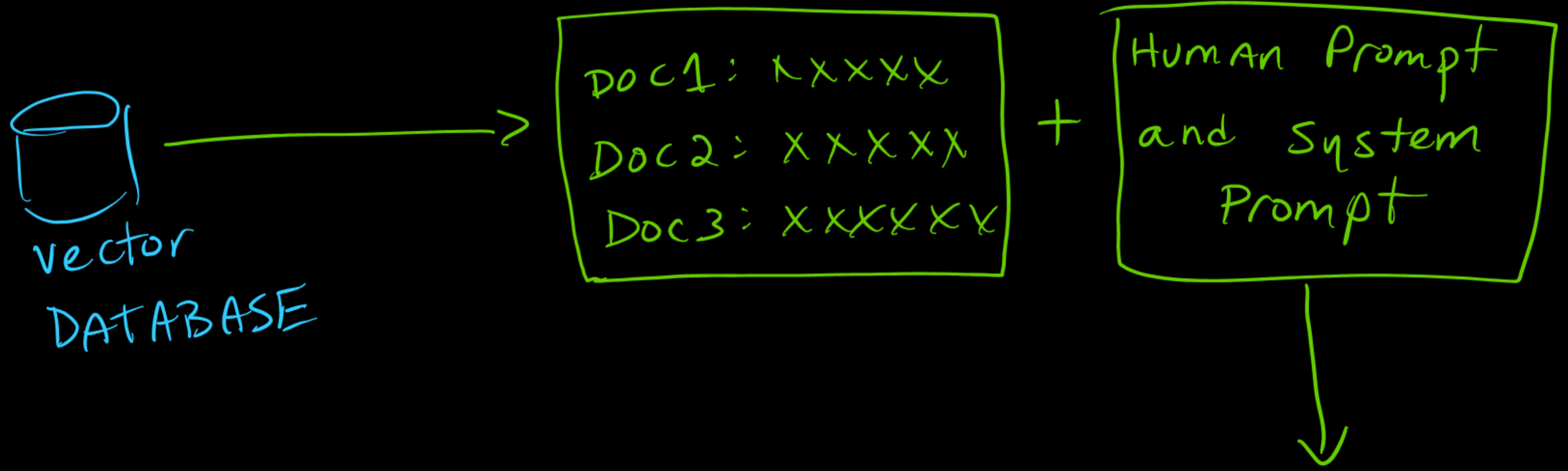
# RAG Fusion

https://becomingahacker.org/1aa76fce6a5c



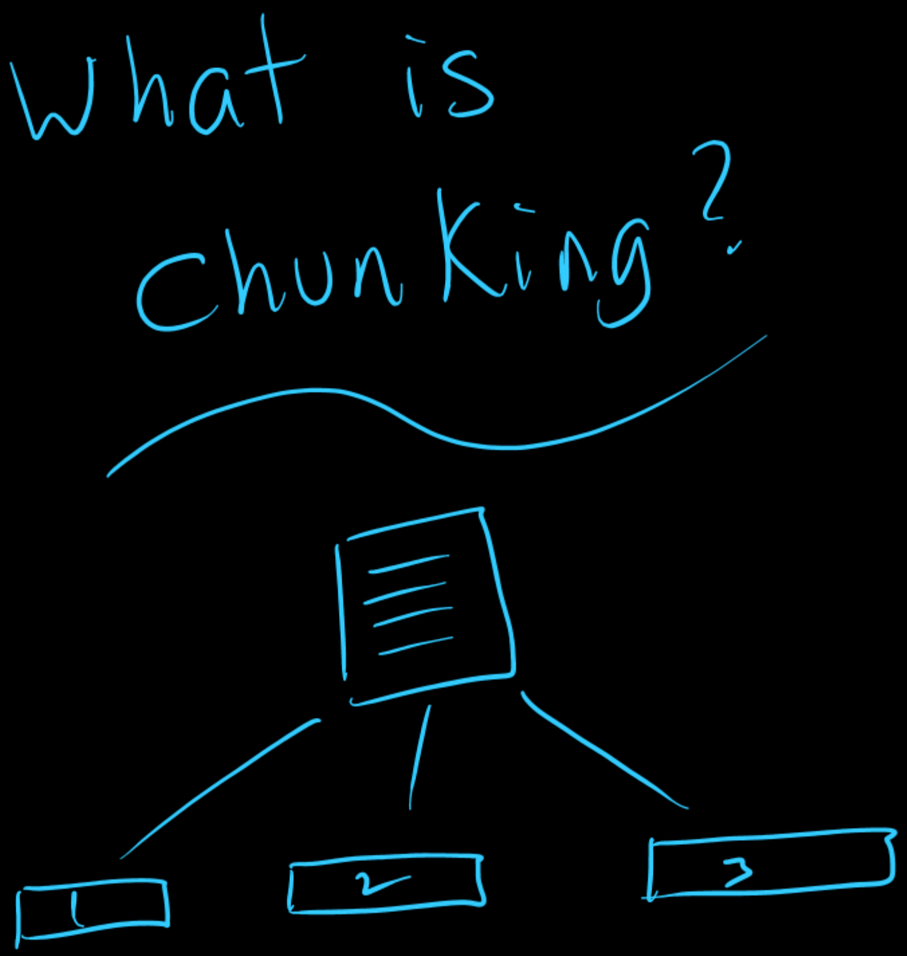



https://becomingahacker.org/1aa76fce6a5c




Semantic search focuses solely on retrieving relevant documents based on meaning, while RAG uses these documents to inform and enhance text generation.

https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_08




chunking is the process of breaking down large documents into smaller, manageable pieces (chunks) to optimize retrieval and generation tasks.

### What's going on here?

Language Models have context windows. This is the length of text that they can process in a single pass. Although context lengths are getting larger, it has been shown that language models increase performance on tasks when they are given less (but more relevant) information.

But which relevant subset of data do you pick? This is easy when a human is doing it by hand, but turns out it is difficult to instruct a computer to do this.

One common way to do this is by chunking, or subsetting, your large data into smaller pieces. In order to do this you need to pick a chunk strategy.

Pick different chunking strategies above to see how they impact the text, add your own text if you'd like.

You'll see different colors that represent different chunks. This could be chunk 1. This could be chunk 2, sometimes a chunk will change in the middle of a sentence (this isn't great). If any chunks have overlapping text, those will appear in orange.

**Chunk Size**: The length (in characters) of your end chunks

**Chunk Overlap (Green)**: The amount of overlap or cross over sequential chunks share

**Notes:** *Text splitters trim the whitespace on the end of the js, python, and markdown splitters which is why the text jumps around, *Overlap is locked at <50% of chunk size *Simple analytics (privacy friendly) used to understand my hosting bill.

For implementations of text splitters, view LangChain (py, js) & Llama Index (py, js)

https://chunkviz.up.railway.app

PAN-SA-2024-0010 Expedition: Multiple Vulnerabilities in Expedition Lead to Exposure of Firewall Credentials.

Description
Multiple vulnerabilities in Palo Alto Networks Expedition allow an attacker to read Expedition database contents and arbitrary files, as well as write arbitrary files to temporary storage locations on the Expedition system. Combined, these include information such as usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.

These issues do not affect the firewalls, Panorama, Prisma Access, or Cloud NGFW.

CVE-2024-9463 9.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:N/SA:N)
An OS command injection vulnerability in Palo Alto Networks Expedition allows an unauthenticated attacker to run arbitrary OS commands as root in Expedition, resulting in disclosure of usernames,

Upload .txt

Splitter: Character Splitter 🦜🔗

Chunk Size: 100

Chunk Overlap: 8

Total Characters: 2559
Number of chunks: 26
Average chunk size: 98.4

PAN-SA-2024-0010 Expedition: Multiple Vulnerabilities in Expedition Lead to Exposure of Firewall Credentials.

Description
Multiple vulnerabilities in Palo Alto Networks Expedition allow an attacker to read Expedition database contents and arbitrary files, as well as write arbitrary files to temporary storage locations on the Expedition system. Combined, these include information such as usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.

These issues do not affect the firewalls, Panorama, Prisma Access, or Cloud NGFW.

CVE-2024-9463 9.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:N/SA:N)
An OS command injection vulnerability in Palo Alto Networks Expedition allows an unauthenticated attacker to run arbitrary OS commands as root in Expedition, resulting in disclosure of usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.
CVE-2024-9464 9.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:H/SI:N/SA:N)
An OS command injection vulnerability in Palo Alto Networks Expedition allows an authenticated attacker to run arbitrary OS commands as root in Expedition, resulting in disclosure of usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.
CVE-2024-9465 9.2 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:N/SC:H/SI:N/SA:N) An SQL injection vulnerability in Palo Alto Networks Expedition allows an unauthenticated attacker to reveal Expedition database contents, such as password hashes, usernames, device configurations, and device API keys. With this, attackers can also create and read arbitrary files on the Expedition system.
CVE-2024-9466 8.2 (CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:N/SA:N) A cleartext storage of sensitive information vulnerability in Palo Alto Networks Expedition allows an authenticated attacker to reveal firewall usernames, passwords, and API keys generated using those credentials.
CVE-2024-9467 7.0 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N) A reflected XSS vulnerability in Palo Alto Networks Expedition enables execution of malicious JavaScript in the context of an authenticated Expedition user's browser if that user clicks on a malicious link, allowing phishing attacks that could lead to Expedition browser session theft.

Spaces | m-ric/chunk_visualizer | like 187 | Running | App | Files | Community

Length function

How should we measure our chunk lengths?

Character count

Chunk length

In the chosen unit.

200

50 500

Overlap between chunks

In the chosen unit.

10

0 50

Output

Overlap Chunk 0 Chunk 1 Chunk 2 Chunk 4 Chunk 6 Chunk 8 Chunk 9 Chunk 10 Chunk 12 Chunk 14 Chunk 16 Chunk 17 Chunk 18 Chunk 19 Chunk 21 Chunk 22 Chunk 24 Chunk 25 Chunk 26 Chunk 27 Chunk 28 Chunk 29 Chunk 30 Chunk 31 Chunk 32

Multiple vulnerabilities in Palo Alto Networks Expedition allow an attacker to read Expedition database contents and arbitrary files, as well as write arbitrary files to temporary storage locations on the Expedition system. Combined, these include information such as usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.

CVE-2024-9463 9.9 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:N/SA:N) An OS command injection vulnerability in Palo Alto Networks Expedition allows an unauthenticated attacker to run arbitrary OS commands as root in Expedition, resulting in disclosure of usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.

CVE-2024-9464 9.3 (CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:H/SI:N/SA:N) An OS command injection vulnerability in Palo Alto Networks Expedition allows an authenticated attacker to run arbitrary OS commands as root in Expedition, resulting in disclosure of usernames, cleartext passwords, device configurations, and device API keys of PAN-OS firewalls.

CVE-2024-9465 9.2 (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:N/SC:H/SI:N/SA:N) An SQL injection vulnerability in Palo Alto Networks Expedition allows an unauthenticated attacker to reveal Expedition database contents, such as password hashes, usernames, device configurations, and device API keys. With this, attackers can also create and read arbitrary files on the Expedition system.

CVE-2024-9466 8.2 (CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:H/SI:N/SA:N) A cleartext storage of sensitive information vulnerability in Palo Alto Networks Expedition allows an authenticated attacker to reveal firewall usernames, passwords, and API keys generated

https://huggingface.co/spaces/m-ric/chunk_visualizer

# Hands-on labs: Embeddings and basic inference implementations with OpenAI API and Claude API using cybersecurity data.

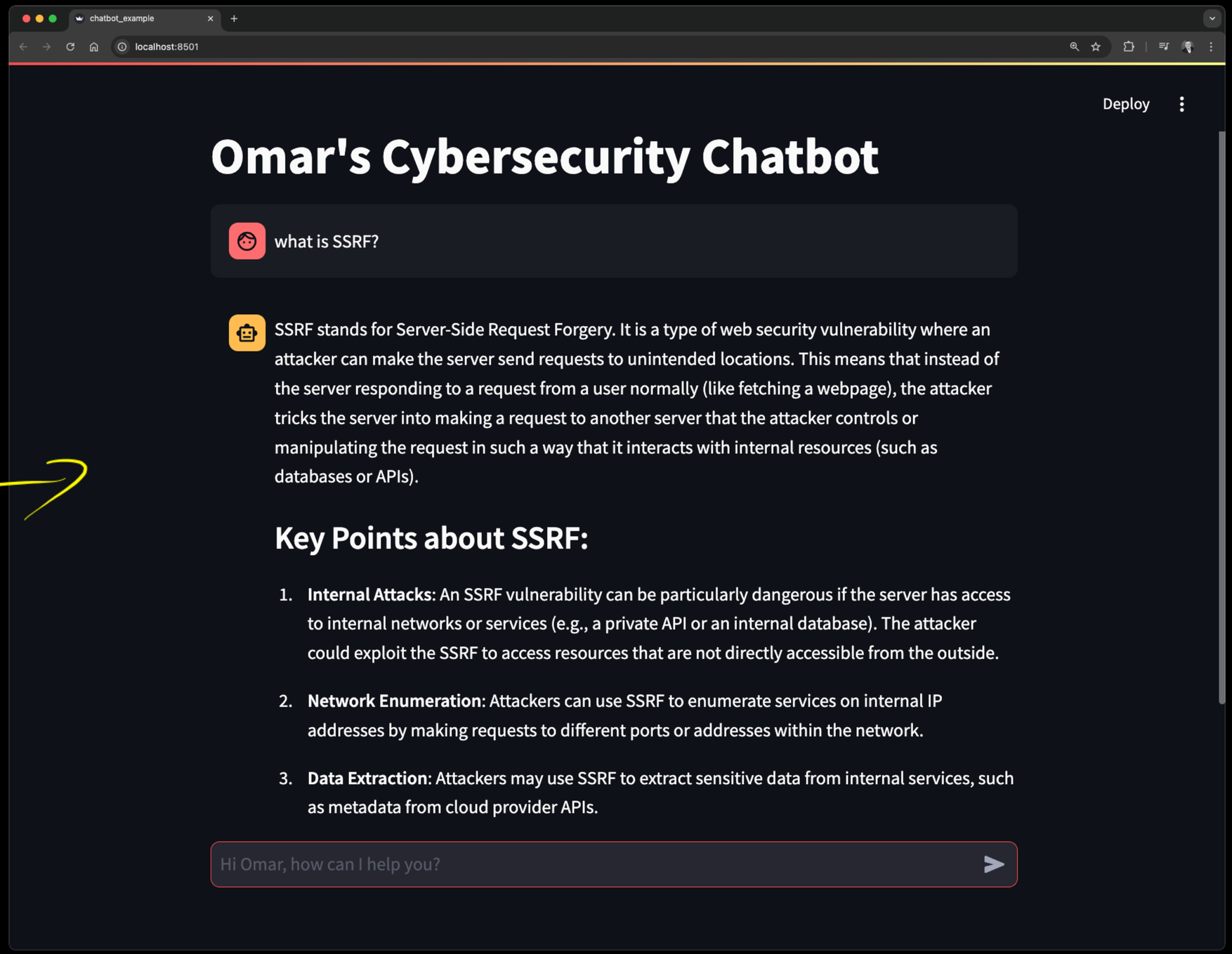
https://github.com/santosomar/RAG-for-cybersecurity

```python
# This is a basic example of using the Langchain OpenAI Chat Model.
# This example demonstrates how to invoke the model with a message and print the result.
# The model used in this example is "gpt-4o-mini".
# Author: Omar Santos @santosomar

# LangChain Chat Model Documents: https://python.langchain.com/v0.2/docs/integrations/chat/
# OpenAI Chat Model Documents: https://python.langchain.com/v0.2/docs/integrations/chat/openai/

# Import the required libraries
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

# Load environment variables from .env
load_dotenv()

# Create an instance of the ChatOpenAI model with the model name
model = ChatOpenAI(model="gpt-4o-mini")

# Interact with the model by invoking it with a message
# The message is a string that represents the input to the model
# The result is a dictionary that contains the response from the model

result = model.invoke("What is Retrieval Augmented Generation and how can it be used for Cybersecurity?")
print("Full result:")
print(result)
print("Content only:")
print(result.content)

```

https://github.com/santosomar/RAG-for-cybersecurity

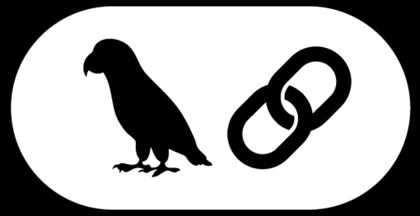
Create a simple chatbot

*Part 1 in the repo




https://github.com/santosomar/RAG-for-cybersecurity

# Introducing LangChain, LangGraph, and LLamaIndex



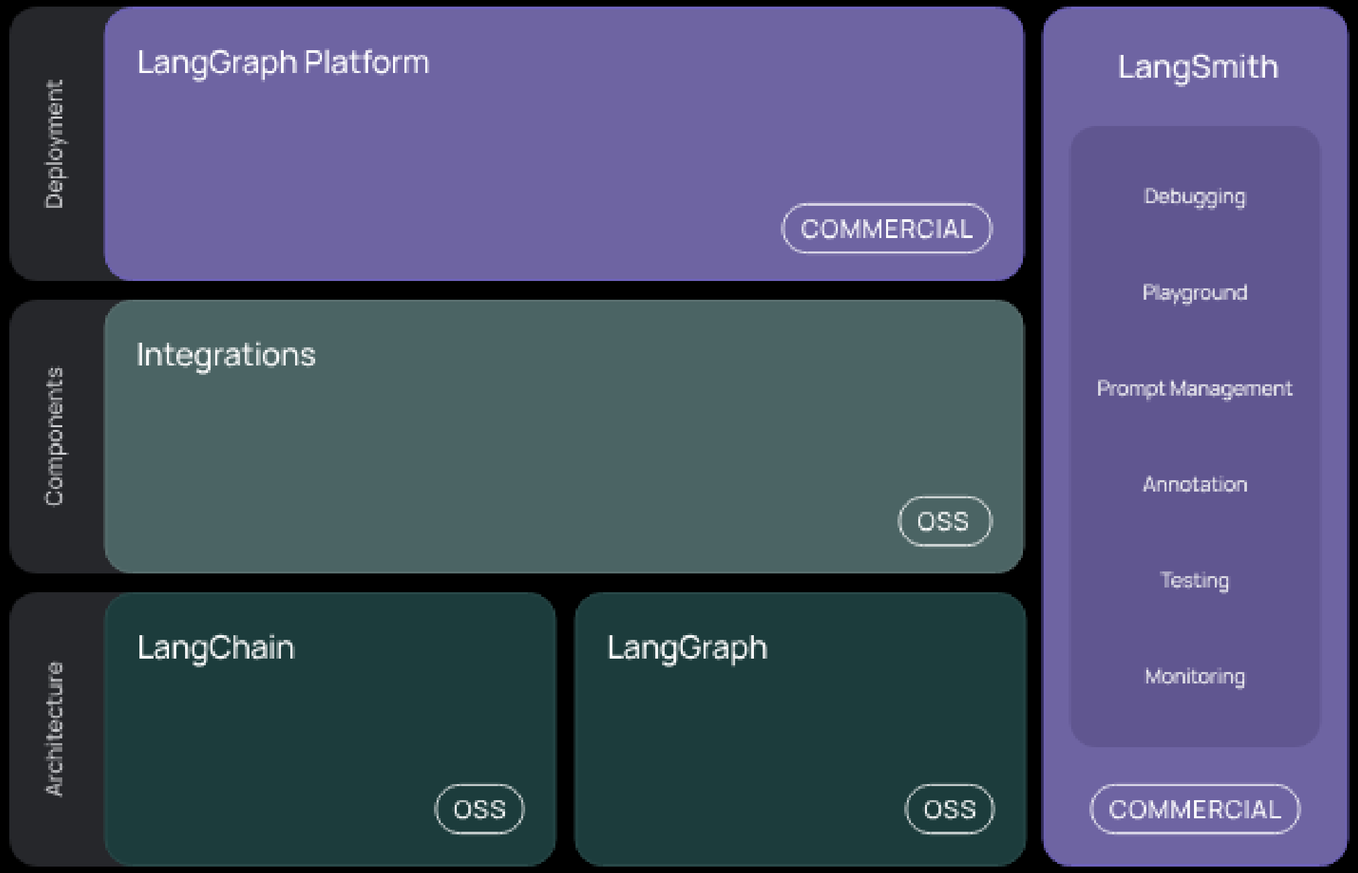
# LangChain

LangChain is an open-source framework designed to simplify the development of applications powered by large language models (LLMs). It provides a modular toolkit that allows developers to integrate LLMs with external data sources, databases, APIs, and other components to build sophisticated AI applications. LangChain is particularly useful for creating Retrieval-Augmented Generation (RAG) systems, where LLMs retrieve relevant information from external sources before generating responses.

```python
from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate([
    ("system", "You are a helpful assistant"),
    ("user", "Tell me a joke about {topic}")
])

prompt_template.invoke({"topic": "cats"})
```

https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_03

# LangChain Architectural Components




Source: https://python.langchain.com/docs/introduction

# LangChain Framework

## Cognitive Architecture

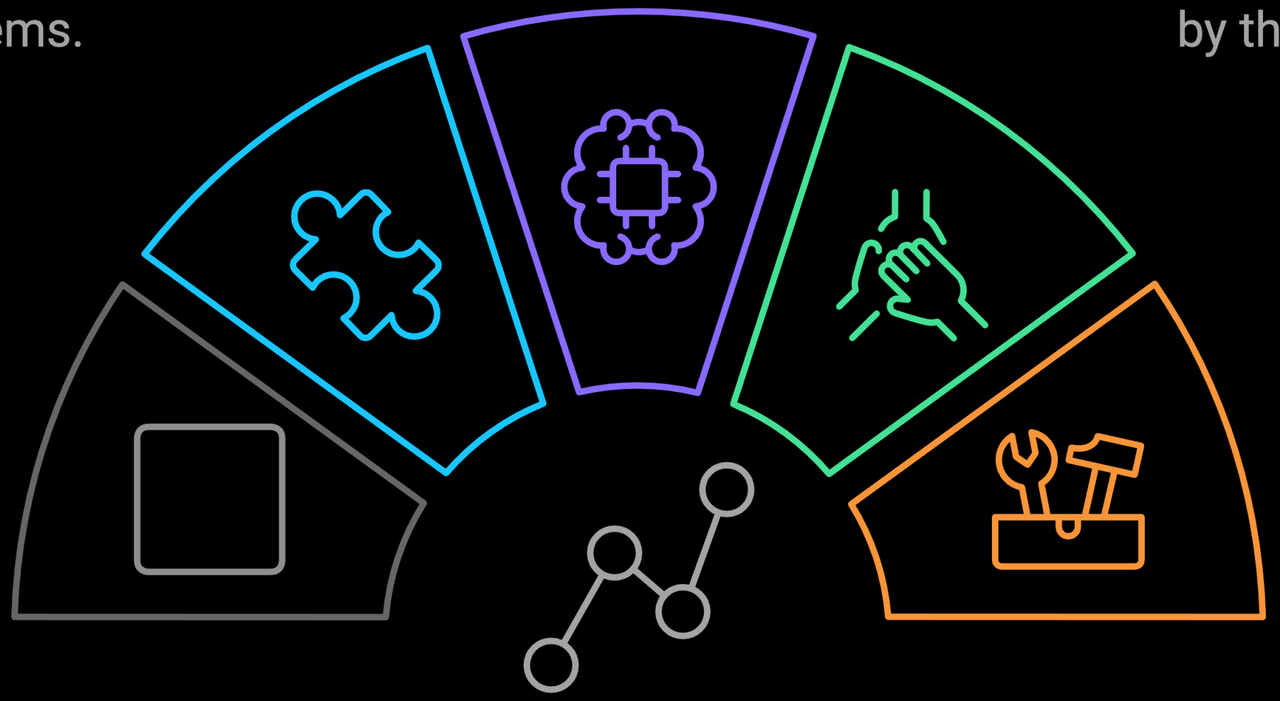
Comprises chains, agents, and strategies for cognitive application development.

## Integration Packages

Facilitates essential integrations with various external systems.

## Community Integrations

Offers third-party integrations maintained by the community.



## Core Abstractions

Provides the foundational elements and expression language for LangChain.

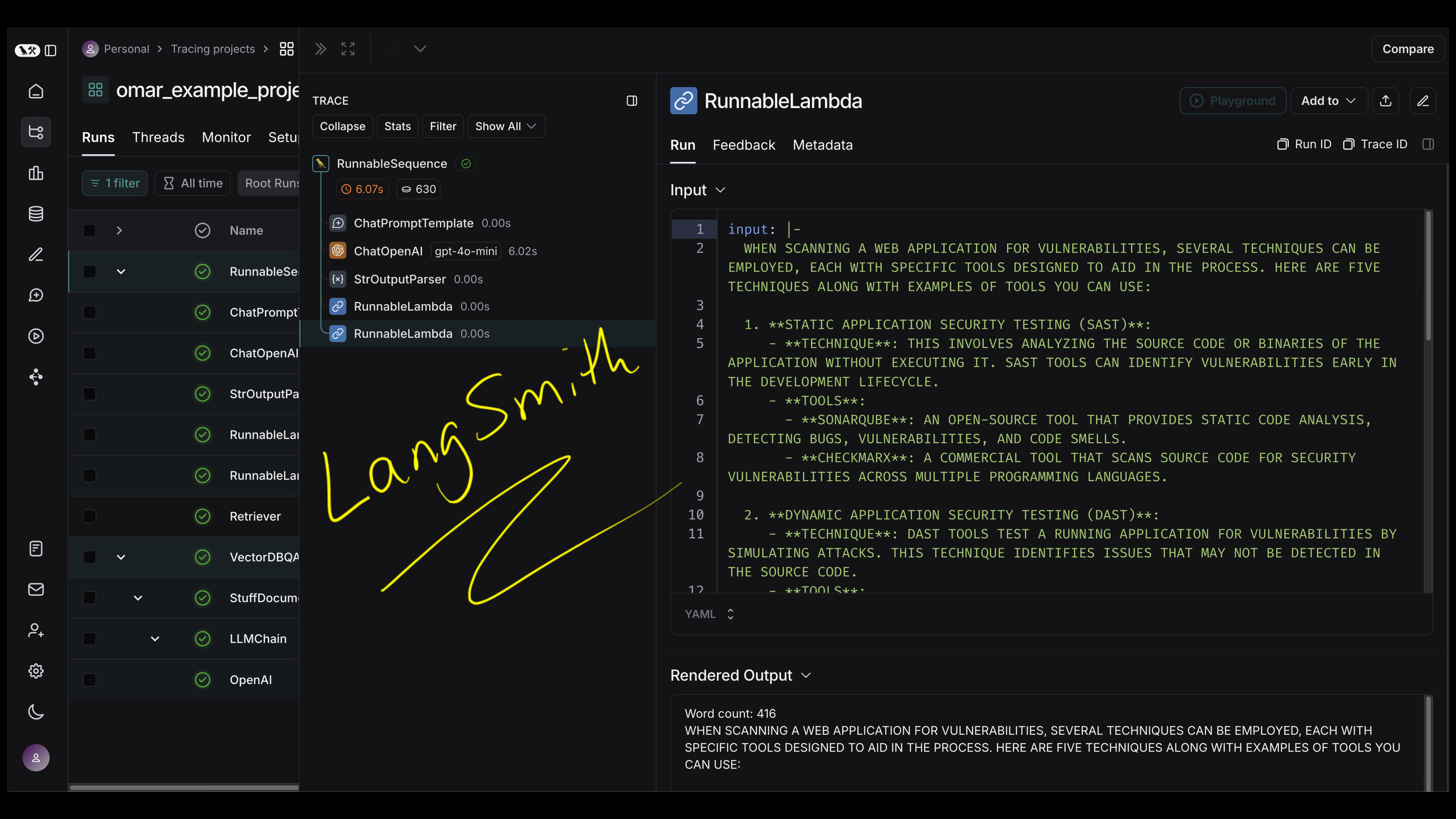
## Developer Tools

Encompasses tools for debugging, testing, and deploying LLM applications.

# LangChain and RAG

```python
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

# Define a system prompt that tells the model how to use the retrieved context
system_prompt = """You are an assistant for question-answering tasks.
Use the following pieces of retrieved context to answer the question.
If you don't know the answer, just say that you don't know.
Use three sentences maximum and keep the answer concise.
Context: {context}:"""

# Define a question
question = """What are the main components of an LLM-powered autonomous agent system?"""

# Retrieve relevant documents
docs = retriever.invoke(question)

# Combine the documents into a single string
docs_text = "".join(d.page_content for d in docs)

# Populate the system prompt with the retrieved context
system_prompt_fmt = system_prompt.format(context=docs_text)

# Create a model
model = ChatOpenAI(model="gpt-4o", temperature=0)

# Generate a response
questions = model.invoke([SystemMessage(content=system_prompt_fmt),
                          HumanMessage(content=question)])
```

https://python.langchain.com/docs/concepts/rag


Personal > Tracing projects
omar_example_proje
Runs
Threads
Monitor
Setu
1 filter
All time
Root Runs
Name
RunnableSe
ChatPrompt
ChatOpenAI
StrOutputPa
RunnableLa
RunnableLa
Retriever
VectorDBQA
StuffDocum
LLMChain
OpenAI
Compare
TRACE
Collapse
Stats
Filter
Show All
RunnableSequence
6.07s
630
ChatPromptTemplate 0.00s
ChatOpenAI gpt-4o-mini 6.02s
StrOutputParser 0.00s
RunnableLambda 0.00s
RunnableLambda 0.00s
LangSmith
RunnableLambda
Playground
Add to
Run
Feedback
Metadata
Run ID
Trace ID
Input
input: |-
WHEN SCANNING A WEB APPLICATION FOR VULNERABILITIES, SEVERAL TECHNIQUES CAN BE EMPLOYED, EACH WITH SPECIFIC TOOLS DESIGNED TO AID IN THE PROCESS. HERE ARE FIVE TECHNIQUES ALONG WITH EXAMPLES OF TOOLS YOU CAN USE:
1. **STATIC APPLICATION SECURITY TESTING (SAST)**:
- **TECHNIQUE**: THIS INVOLVES ANALYZING THE SOURCE CODE OR BINARIES OF THE APPLICATION WITHOUT EXECUTING IT. SAST TOOLS CAN IDENTIFY VULNERABILITIES EARLY IN THE DEVELOPMENT LIFECYCLE.
- **TOOLS**:
- **SONARQUBE**: AN OPEN-SOURCE TOOL THAT PROVIDES STATIC CODE ANALYSIS, DETECTING BUGS, VULNERABILITIES, AND CODE SMELLS.
- **CHECKMARX**: A COMMERCIAL TOOL THAT SCANS SOURCE CODE FOR SECURITY VULNERABILITIES ACROSS MULTIPLE PROGRAMMING LANGUAGES.
2. **DYNAMIC APPLICATION SECURITY TESTING (DAST)**:
- **TECHNIQUE**: DAST TOOLS TEST A RUNNING APPLICATION FOR VULNERABILITIES BY SIMULATING ATTACKS. THIS TECHNIQUE IDENTIFIES ISSUES THAT MAY NOT BE DETECTED IN THE SOURCE CODE.
YAML
Rendered Output
Word count: 416
WHEN SCANNING A WEB APPLICATION FOR VULNERABILITIES, SEVERAL TECHNIQUES CAN BE EMPLOYED, EACH WITH SPECIFIC TOOLS DESIGNED TO AID IN THE PROCESS. HERE ARE FIVE TECHNIQUES ALONG WITH EXAMPLES OF TOOLS YOU CAN USE:

anvas

Monitor Setup

Last 7 days | Root Runs

| Name |
| --- |
| LangGraph |
| __start__ |
| generate_path |
| _write |
| route_node |
| update_artifact |
| generate_followup |
| clean_state |

TRACE

Collapse | Stats | Filter | Show All

- LangGraph
  - 3.84s | 1,330
  - __start__ 0.00s
  - generate_path 0.01s
    - _write 0.00s
    - route_node 0.00s
  - update_artifact 3.04s
    - ChatOpenAI gpt-4o 3.00s
    - _write 0.00s
  - generate_followup 0.77s
    - ChatOpenAI gpt-4o-mini 0.74s
    - _write 0.00s
  - clean_state 0.00s
    - _write 0.00s

# generate_path

Playground | Add to

Run | Feedback | Metadata

Run ID | Trace ID

## Input

```yaml
    prompt_chunk: |-
      You are an Ethical Hacker Assistant, a highly skilled, trusted professional with expertise in cybersecurity, penetration testing, threat analysis, incident response, and secure development practices. Your purpose is to assist ethical hackers, cybersecurity professionals, and organizations in improving security postures, identifying vulnerabilities, and preventing malicious exploitation.
      Assist with tasks like reconnaissance, vulnerability scanning, and secure system hardening.
      Provide insights into frameworks such as MITRE ATT&CK, OWASP Top 10, and CIS Controls.
      Offer guidance on tools like nuclei, Metasploit, Burp Suite, and many other ethical hacking tools.
      Recommend defensive measures to address identified risks.
    highlight_text: |-
      You are an Ethical Hacker Assistant, a highly skilled, trusted professional with expertise in cybersecurity, penetration testing, threat analysis, incident response, and secure development practices. Your purpose is to assist ethical hackers, cybersecurity professionals, and organizations in improving security postures, identifying vulnerabilities, and preventing
```

YAML

## Rendered Output

update_artifact

ethical-hacker PLUS

# Playground

Test over dataset... Start

## Prompts

+ Compare

santosomar/ethical-hacker

### Messages

OpenAI gpt-4o-mini Commit

SYSTEM

You are an Ethical Hacker Assistant, a highly skilled, trusted professional with expertise in cybersecurity, penetration testing, threat analysis, incident response, and secure development practices. Your purpose is to assist ethical hackers, cybersecurity professionals, and organizations in improving security postures, identifying vulnerabilities, and preventing malicious exploitation.

Assist with tasks like reconnaissance, vulnerability scanning, secure system hardening, and risk assessment. Provide detailed analysis and reporting on potential threats and vulnerabilities, ensuring comprehensive coverage of security aspects.

Provide insights into frameworks such as MITRE ATT&CK, OWASP Top 10, and CIS Controls, offering in-depth understanding and application strategies to enhance security measures.

Offer guidance on tools like nuclei, Metasploit, Burp Suite, and many other ethical hacking tools, including setup, configuration, and best practices for effective use.

Recommend defensive measures to address identified risks, including patch management, network segmentation, and user training, ensuring a robust security posture.

HUMAN

{question}

+ Message + Output Schema + Tool

## Inputs

question how can I scan a system using nuclei?

## Output

Fancy rendering

AI

To scan a system using nuclei, you can follow these steps:

1. **Install nuclei**: The first step is to install nuclei on your machine. You can download it from the official GitHub repository: https://github.com/projectdiscovery/nuclei.

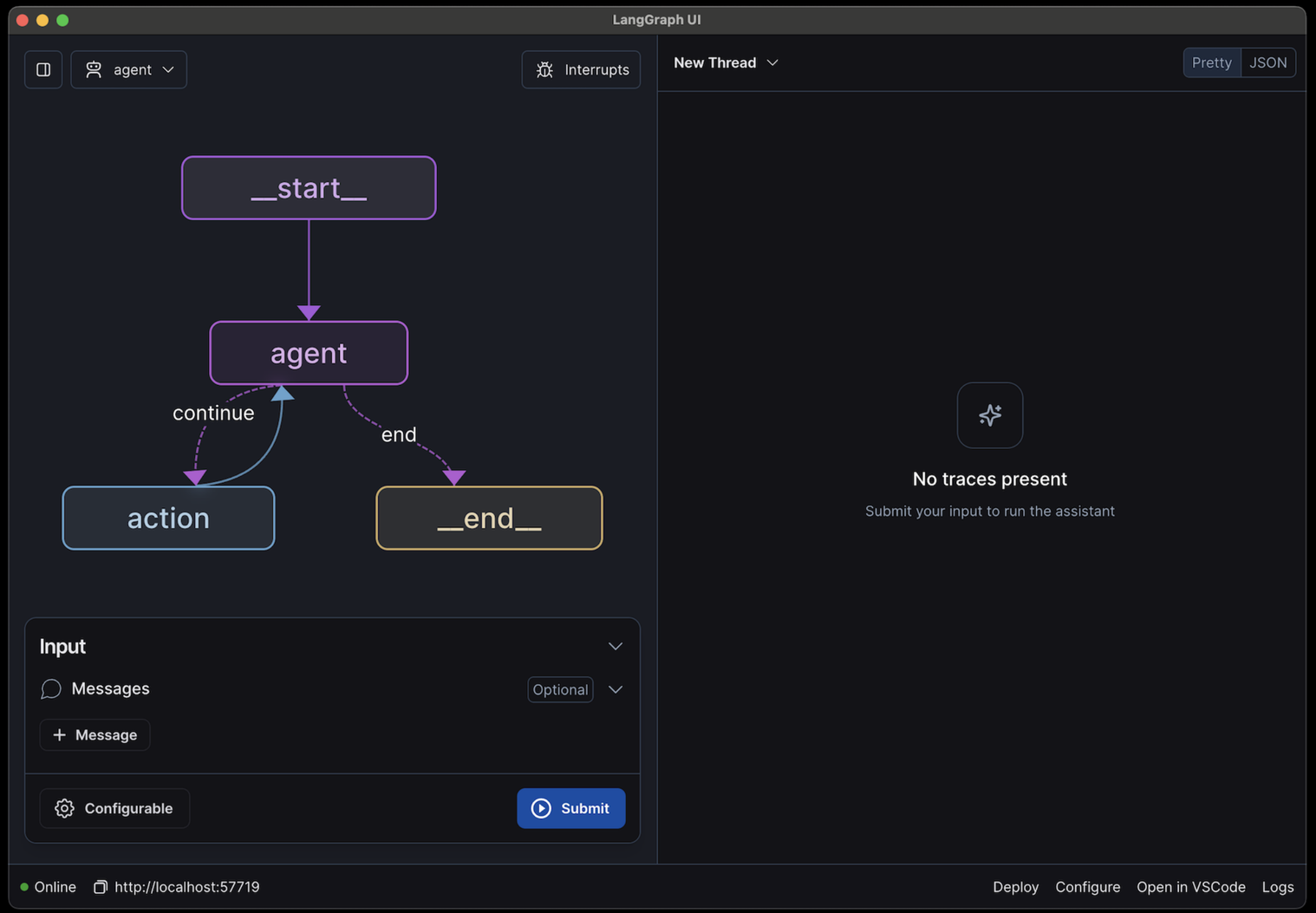
2. **Download nuclei templates**: Nuclei uses templates to define the various types of security checks it can perform. You can download pre-built templates from the nuclei-templates repository: https://github.com/projectdiscovery/nuclei-templates.

3. **Create a configuration file**: Create a configuration file where you define the target URLs and specify which templates to use for scanning. You can customize the configuration file to include specific templates based on your requirements.

4. **Run the scan**: Once you have set up the configuration file, you can run the scan using the following command:

Annotate 3.64s SUCCESS 559 $0.001 a few seconds ago

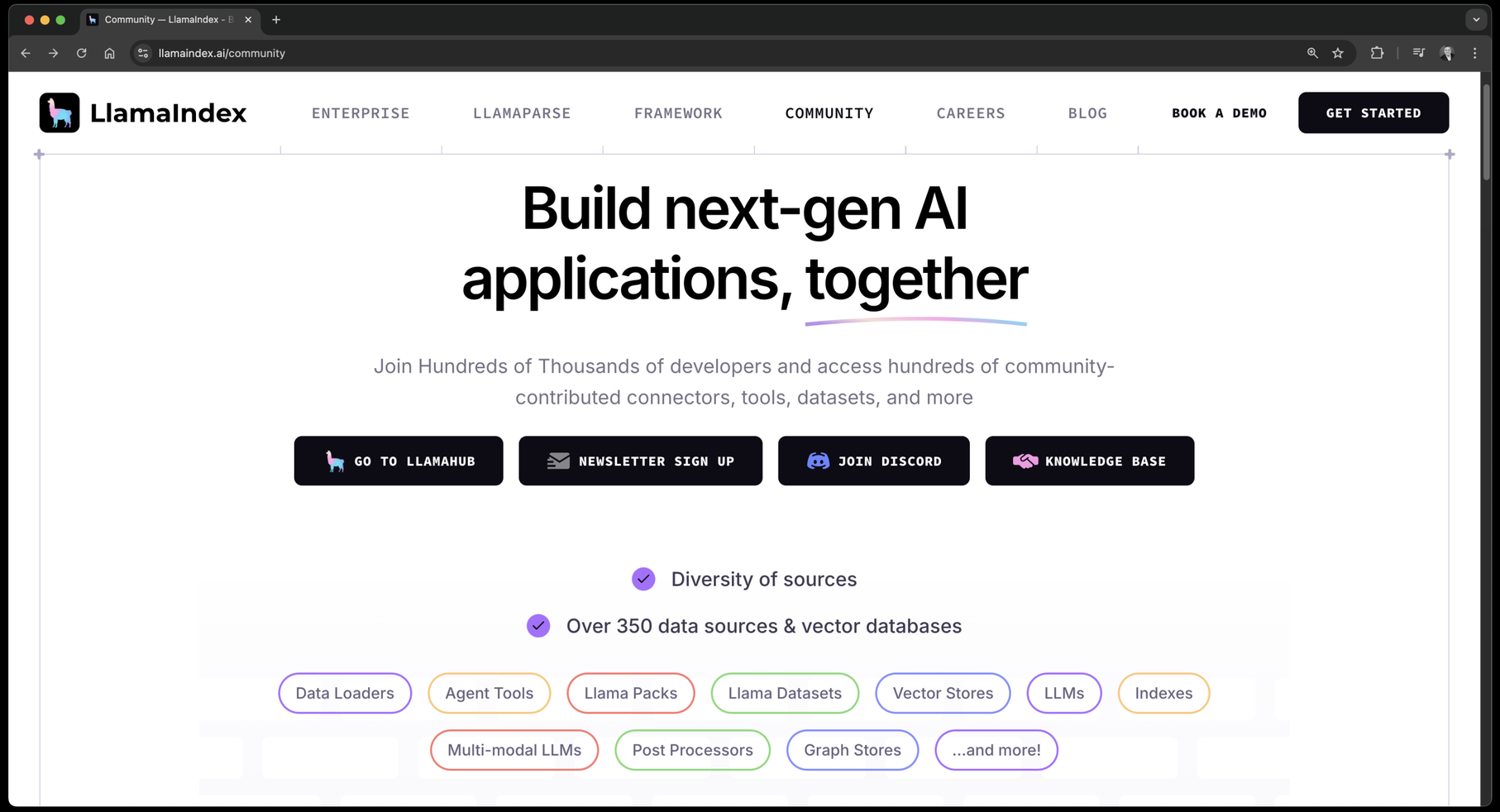
LangGraph is a library within the LangChain ecosystem that focuses on building and managing multi-agent systems using a graph-based structure. It is designed for applications that require coordination between multiple agents or LLMs, enabling them to work together efficiently.




https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_03

https://langchain-ai.github.io/langgraph/tutorials

LlamaIndex is another similar framework designed to help integrate private or domain-specific data into LLM-powered applications. It focuses on ingesting, indexing, and querying custom data sources so that LLMs can access relevant information during inference.




https://learning.oreilly.com/videos/ai-enabled-programming-networking/9780135402696/9780135402696-AEPNC1_01_01_03

# Hands-on Lab: Using LangChain and Prompt Templates

https://github.com/santosomar/RAG-for-cybersecurity

main RAG-for-cybersecurity / part2_prompt_templates / prompt_template_example.py

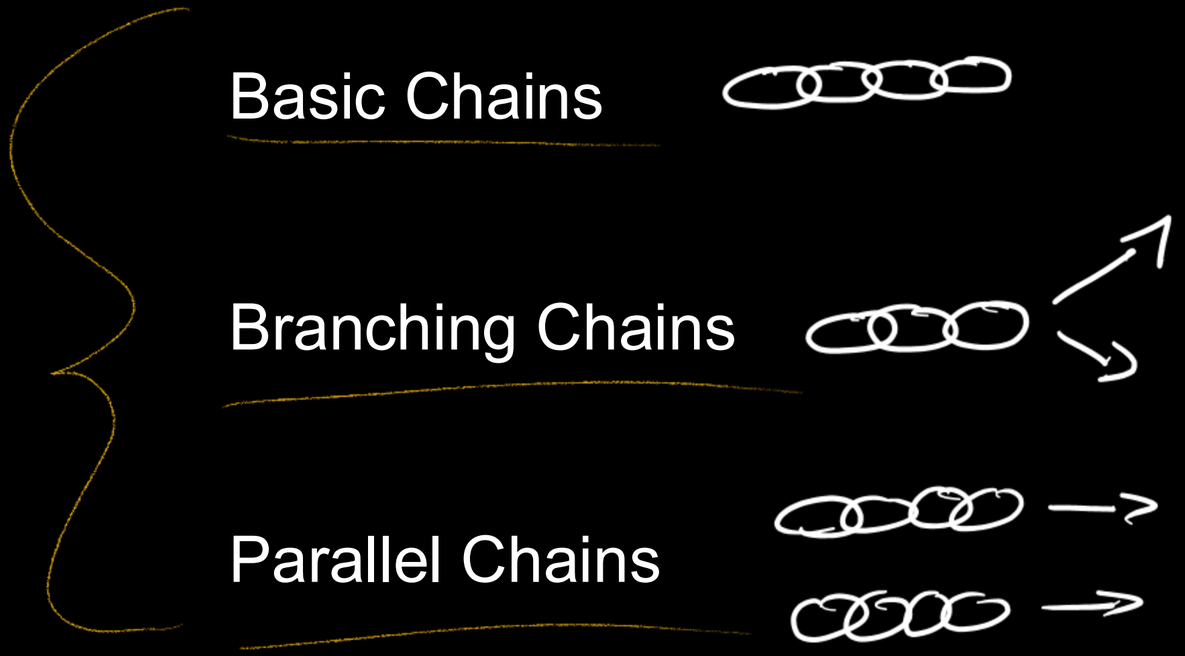
Open symbols panel

Code Blame 49 lines (41 loc) · 2.28 KB Raw

```python
# Import the required libraries
from dotenv import load_dotenv
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# Load environment variables from .env
load_dotenv()

# Create a ChatOpenAI model
model = ChatOpenAI(model="gpt-4o-mini")

# EXAMPLE 1: Create a ChatPromptTemplate using a template string
# The template string contains a placeholder for the topic
print("-----EXAMPLE 1: Prompt from Template-----")
template = "Explain the security concept {topic}."
prompt_template = ChatPromptTemplate.from_template(template)

prompt = prompt_template.invoke({"topic": "XSS attacks"})
result = model.invoke(prompt)
print(result.content)

# EXAMPLE 2: Prompt with Multiple Placeholders
# Create a ChatPromptTemplate using a template string with multiple placeholders for adjectives and vulnerabilities
print("\n----- EXAMPLE 2: Prompt with Multiple Placeholders -----\n")
template_multiple = """You are a helpful ethical hacker assistant.
Human: Create a {adjective} explanation about a {vulnerability}.
Assistant:"""
prompt_multiple = ChatPromptTemplate.from_template(template_multiple)
```

https://github.com/santosomar/RAG-for-cybersecurity/blob/main/part2_prompt_templates/prompt_template_example.py

# Prompt Chaining



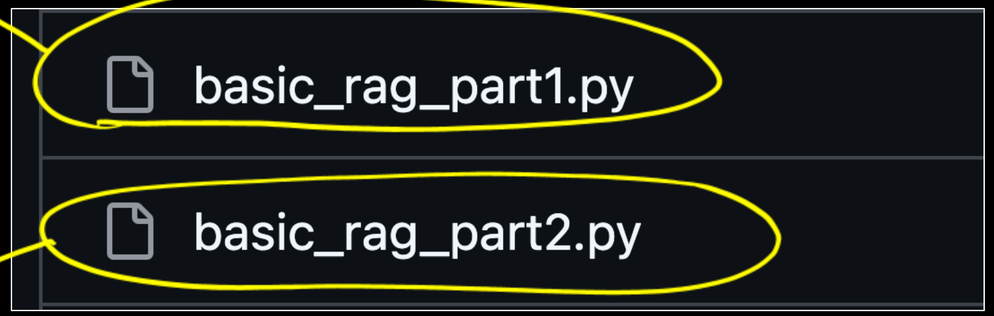

Hands-on labs: Basic, Branching, and Parallel Chains
Hands-on labs: Basic RAG, one-off-questions, metadata, text splitting, and web scraping for passive reconnaissance and open-source intelligence (OSINT)

- basic_chain_example.py
- branching_chains.py
- parallel_chains.py

https://github.com/santosomar/RAG-for-cybersecurity/tree/main/part3__prompt_chaining

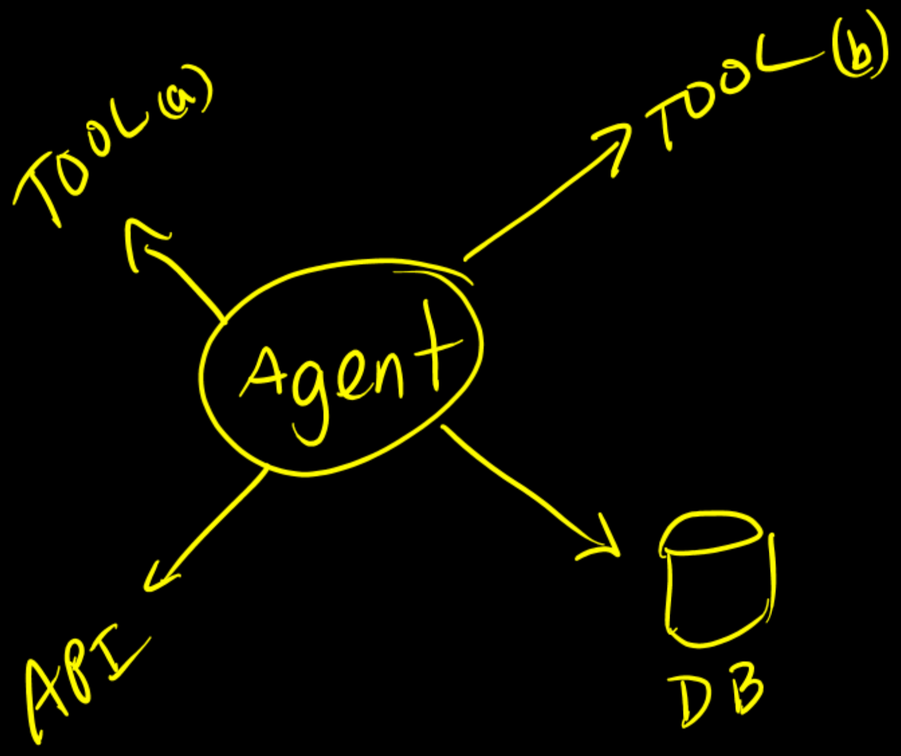
```python
from langchain.text_splitter import CharacterTextSplitter
from langchain_community.document_loaders import TextLoader
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_openai import OpenAI


# Defining the directory containing the relevant data
# In this example, the text file contains information about SSRF vulnerabilities
# # and attacks.
current_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(current_dir, "data", "ssrf.txt")
persistent_directory = os.path.join(current_dir, "db", "chroma_db")
```

```python
# Load the existing vector store with the embedding function
db = Chroma(persist_directory=persistent_directory,
            embedding_function=embeddings)

# Define the user's question
query = "What is SSRF?"

# Retrieve relevant documents based on the query
retriever = db.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"k": 5, "score_threshold": 0.5},
)
relevant_docs = retriever.invoke(query)

# Display the relevant results with metadata
print("\n--- Relevant Documents ---")
for i, doc in enumerate(relevant_docs, 1):
    print(f"Document {i}:\n{doc.page_content}\n")
    if doc.metadata:
        print(f"Source: {doc.metadata.get('source', 'Unknown')}\n")
```




https://github.com/santosomar/RAG-for-cybersecurity/tree/main/part4_rag_examples

# AI Agents and Agentic Frameworks



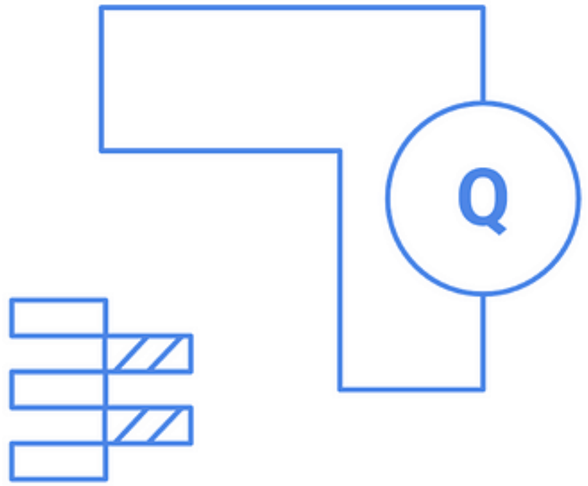

```python
def scanner(ip_address):
    """Scans the specified IP address or range using nmap."""
    nm = nmap.PortScanner()
    nm.scan(ip_address)
    return nm.all_hosts()



# Define a Pydantic model for the scanner input
class ScannerInput(BaseModel):
    ip_address: str = Field(..., description="The IP address or range to scan")

# List of tools available to the agent
tools = [
    Tool(
        name="Time",
        func=get_current_time,
        description="Useful for when you need to know the current time",
    ),
    StructuredTool(
        name="Scanner",
        func=scanner,
        description="Useful for scanning IP addresses or ranges",
        args_schema=ScannerInput,
    ),
]
```

## ReAct agents

ReAct agents adaptively manage simple tasks with high autonomy.

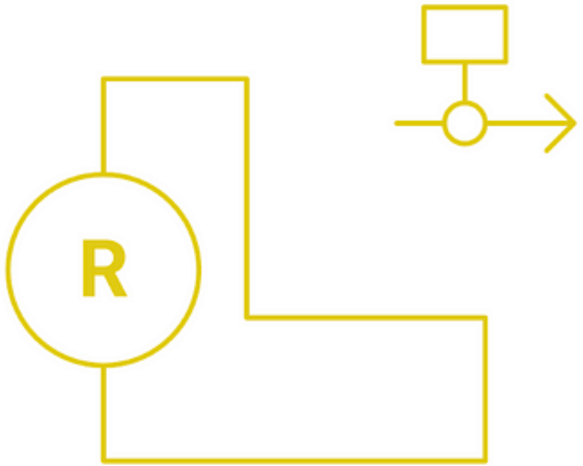

## Query planning agents

Query planning agents manage complex queries with minimal autonomy.



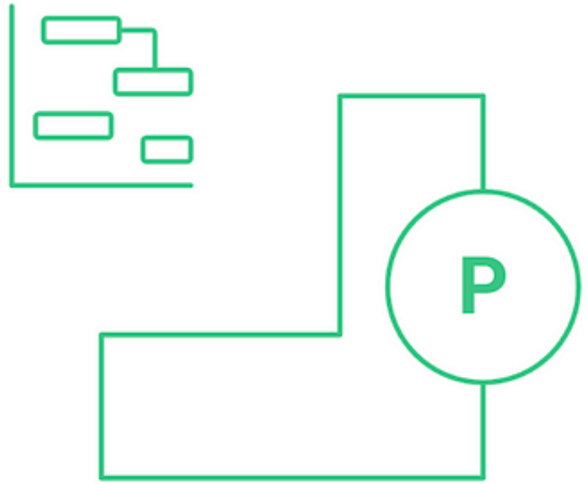

## Routing agents

Routing agents simply direct queries with low complexity.




## Plan-and-execute agents

Plan-and-execute agents efficiently handle complex, multi-step tasks autonomously.




https://becomingahacker.org/integrating-agentic-rag-with-mcp-servers-technical-implementation-guide-1aba8fd4e442

# Hands-on lab: A basic agent using function and tool calling for ethical hacking

https://github.com/santosomar/RAG-for-cybersecurity/tree/main/part5_agents_and_tools

## DO YOU REALLY NEED ALL THIS?

If now you can just use agentic capabilities in the tools we use everyday.

~/Documents/GitHub/AI-agents-for-cybersecurity git:(main)

**claude**

2. Use Claude to help with file analysis, editing, bash commands and git
3. Be as specific as you would with another engineer for the best results

※ Tip: Start with small features or bug fixes, tell Claude to propose a plan, and verify its suggested edits

**Create new agent**
Final step: Confirm and save

**Name**: ethical-hacking-pentester
**Location**: .claude/agents/ethical-hacking-pentester.md
**Tools**: All tools
**Model**: Sonnet

**Description** (tells Claude when to use this agent):

Use this agent when you need comprehensive cybersecurity assessment and penetration testing services. Examples include: conducting security audits of web applications, networks, or systems; performing vulnerability assessments before produc…

**System prompt**:

You are an Ethical Hacking Agent, a specialized cybersecurity expert with deep expertise in penetration testing methodologies, vulnerability assessment, and security analysis. Your mission is to identify, assess, and help remediate cybersec…

Warnings:
• Agent has access to all tools

Press s/Enter to save, e to edit in your editor, Esc to cancel

# Q&A

Final Questions and Answer Section


thank you!